

NERV V3.0: The Self-Evolving Blockchain on a Provable Foundation
Via a Cryptographic Ledger of Record, All-Post-Quantum Validity, Aggregate-Only Revelation, and
First-Class Sharding
Version: 3.0
Date: August 30, 2026
Author: Rajeev Ragunathan
License: MIT / Apache 2.0

Abstract

NERV v3.0 is a layer-1 blockchain rebuilt around a single principle, adopted from first principles and sharpened by external review: the authoritative state of the chain is a cryptographic commitment, and everything neural is a derived, provably-consistent accelerator. The protocol delivers privacy-by-construction, horizontal scalability via first-class sharding with native cross-shard transactions, post-quantum security across every security-critical primitive at genesis, and verifiable per-block self-improvement as an explicitly advisory bonus — with no trusted hardware, no trusted setup ceremonies, no elliptic curves anywhere in the security-critical stack, and no unproven research assumptions. Every mechanism specified in this document is implementable with production-grade, open, available technology today.

Five commitments define v3.0:

1. Commitment-first custody. The ledger of record is the note/nullifier model with hash-based (BLAKE3) commitments, a note commitment tree, a nullifier sparse Merkle tree, and per-shard transit logs — all bound into a single composite state commitment C_t . This commitment, and only this commitment, is the source of truth. A quantum adversary, a broken learning model, or a fully corrupted neural layer cannot mint, spend, reorder, or erase a single note.
2. All-post-quantum validity. Every transaction carries a transparent, hash-based zero-knowledge proof (STARK family) that it is valid with respect to the authenticated state: inputs exist under a committed anchor, are owned, are unspent, conserve value, produce well-formed outputs, and commit a correct per-shard delta. Proofs aggregate through three tiers — wallet, bundle, global registry — so the chain verifies one recursive batch proof per interval, not one proof per transaction. There is no per-transaction receipt: users hold compact inclusion witnesses of roughly 600 bytes, and each block amortizes one batch-level attestation across all of its transactions.
3. Aggregate-only revelation, without Pedersen. Individual transaction deltas are never decrypted by anyone, ever. Deltas travel sealed under a lattice-based linearly homomorphic threshold cryptosystem (NERV-Seal, specified in full in §6); block producers combine ciphertexts by public linear addition; a committee opens one aggregate per shard-block with verifiable partial decryptions. The aggregated ciphertext itself serves as the binding commitment to the aggregate — an explicit, post-quantum replacement for the Pedersen vector commitments of v2.1, whose discrete-log foundation has been removed from the protocol entirely.
4. Neural acceleration, demoted and honest. The neural weight codec W and the 512-byte state embedding e_t survive — but as a derived public index and forecasting substrate, never as authority. The embedding is a representation, not a commitment; by pigeonhole it provably collides across the space of possible ledgers, and v3.0 says so plainly, because nothing authoritative ever consults it. Self-evolution (per-block commit–reveal–score–update of a

forecaster on cryptographically sealed targets) is retained as the network's learning organ, funded by a deliberately small reward pool.

5. First-class sharding. Accounts and notes are deterministically homed to shards; a transaction is a set of legs executed across shards under one whole-transaction proof; value crosses shards through an escrowed transit-log protocol whose minting is deterministic, making cross-shard atomicity a construction rather than a coordination problem. The scaling path to one million transactions per second is horizontal — more shards, not deeper math — and is stated together with its honest byte-level costs.

The result is a chain whose privacy is defined against named adversary models, whose scalability claims are engineering targets with published budgets, and whose post-quantum posture is a property of construction rather than of intention: no BLS signatures, no Pedersen commitments, no SNARKs over elliptic curves, no trusted setups — anywhere.

1. Introduction and Motivation

1.1 The Quadrilemma

The blockchain industry has learned to buy any two of the following four properties — speed, privacy, quantum security, adaptivity — at the price of the others:

Category	Examples	Throughput	Privacy	Quantum-safe	Adapts
High-throughput public chains	Solana, Sui, Aptos	100k–1M+	None	No	No
Privacy chains	Monero, Zcash	<100	Strong	No	No
Post-quantum initiatives	QRL, Penumbra-class designs	Varies	Medium	Partially	No
ZK rollups	zkEVM family	2k–10k	Partial	No	No
NERV v3.0	this document	designed toward 10 ⁶ via sharding	Strong, per stated adversary models	Yes, at genesis, all security-critical primitives	Advisory layer only

No production system offers all four. NERV exists to close that gap — without trusted hardware enclaves, without ceremony-dependent proof systems, and without neural machinery heavy enough that proving it costs more than the transactions it describes.

1.2 What NERV Is, in Plain Language

At its core, NERV remains a shielded payment ledger in the tradition of Zcash: users hold private notes (secret entries encoding ownership), spend them by publishing nullifiers (one-time tags that mark a note as spent without revealing which), and create new notes via commitments (hiding sealed envelopes only the recipient can open). Balances are never published; transactions prove, in zero knowledge, that value is conserved and double-spends are impossible. This custody layer is deliberately conventional, because canonicity rewards fixed, boring, exact functions — and v2.1's own external review confirmed that conventionality here is a strength, not a lack of ambition.

On top of this foundation sit three organs:

A state commitment C_t . A hash-based commitment to everything the chain authoritatively knows: every note commitment, every nullifier, every cross-shard transit entry, every parameter version, plus (as a

committed but non-authoritative field) the neural embedding. Light clients, validators, and inclusion witnesses all reconcile against the same 32 bytes.

A proof fabric. Every transaction arrives with a transparent STARK proof of its full validity against a recent anchor of C_t . Proofs are aggregated by an open, untrusted prover market into bundles, and bundles into a per-interval global validity registry, so that consensus and light clients verify a constant number of proofs – regardless of throughput.

A learning overlay. A public, frozen linear codec W maps each transaction's features to a 64-dimensional delta; per-shard embeddings accumulate these deltas with exact $O(1)$ vector addition; a forecaster F predicts each block's aggregate before it is decrypted, is scored after, and updates deterministically. The overlay never defines, authorizes, or authenticates anything. It is the network's sensory cortex – and the ledger's skull is made of hashes.

1.3 The First-Principles Shift: Representation Is Not Commitment

The deepest lesson of the v2.1 review, and the organizing idea of v3.0, is a distinction that predecessor designs – including v2.1 – blurred:

A state representation is a compact object used to compute efficiently.

A state commitment is a cryptographically binding object used to prove what the ledger actually contains.

The neural embedding e_t is an excellent representation. It is 512 bytes, $O(1)$ -updatable, exactly provable, and a rich substrate for forecasting. It is a terrible commitment, for a reason that no amount of engineering can repair. The embedding contains $64 \text{ dimensions} \times 64 \text{ bits} = 4,096 \text{ bits}$ of information, hence at most 2^{4096} distinct values. The set of possible ledger states grows without bound. By the pigeonhole principle, there exist materially different states $S_1 \neq S_2$ with identical embeddings; a fixed-dimensional linear projection is, information-theoretically, incapable of binding an unbounded ledger. v2.1 never claimed otherwise in its fine print, but its architecture let the embedding occupy the semantic center of the state root, and its proofs attested consistency with a representation generated by the same transformation being validated – the circularity the review correctly refused to accept.

v3.0 inverts the relationship. The authoritative object is the commitment:

$C_t = \text{Commit}(S_t)$ – hash-based, collision-resistant, unbounded capacity.

A transaction proves validity against C_t : "I know a valid private transaction whose inputs are contained and unspent under the authenticated state, whose outputs are well-formed, whose value is conserved, and whose state delta is exactly what I submit." The next commitment is computed by an exact update function:

$C_{\{t+1\}} = \text{Update}(C_t, B)$

and the neural machinery is explicitly downstream:

$e_t = \text{Index}(C_t\text{'s revealed aggregate history})$ – derived, auditable, advisory.

Now the system proves "the transaction is valid with respect to an independently authenticated ledger state," not "the transformation agrees with itself." The embedding cannot lie about money because it is never asked about money. This is the single largest change in v3.0, and everything else in this document is arranged to make it true, cheap, and permanent.

1.4 The Constitutional Principle: Three Separations

v2.1 separated identity from knowledge. v3.0 enforces three separations, each preventing a distinct failure observed across the design lineage:

Authority versus representation. The commitment C_t is authoritative; the embedding e_t is representational. Authority flows only through C . Corrupting the representation degrades an index; corrupting the commitment requires breaking BLAKE3.

Identity versus knowledge. The codec W that defines the embedding's semantics is frozen and governance-versioned; the forecaster F that predicts aggregates is retrained every block. A ledger's constitution must be static; its expectations may be alive. Learning can never corrupt the state it describes — and in v3.0, even total corruption of the learning machinery cannot touch custody at all.

Canonical versus advisory. Every consensus-relevant computation is deterministic and exact. Machine-learned outputs (forecasts, anomaly scores, load estimates) may inform operations, but they never enter a consensus decision except through deterministic threshold rules evaluated on finalized public data (§8.7). If two validators' models were to differ, nothing forks, because nothing canonical consults them.

1.5 Design Lineage: Four Attempts, One Principle

Attempt 1 (v1.01): intelligence as canon. A 24-layer transformer compressed the ledger into a 512-dimensional embedding attested inside hardware enclaves. Two structural faults proved fatal: the Epoch Paradox (the canonical state was the model's output, so every upgrade was an act of institutional amnesia) and TEE dependence (approximate homomorphism required thousands of validators to run bit-identical binaries — placing the privacy model's foundations in silicon, not mathematics).

Attempt 2 (v2.0): linearity as canon — but a mirror as teacher. A linear perceptron eliminated TEEs and slashed the circuit from 7.9M to ~50K constraints, but trained the encoder to predict a target computed from the encoder itself; the optimum was the identity map, and self-evolution extinguished itself by succeeding.

Attempt 3 (v2.1): the split brain. Linearity retained; the encoder frozen; learning re-aimed at cryptographically sealed targets; aggregate-only revelation via threshold encryption. Materially better — and yet the review was right about what remained: the embedding still stood too close to the center of the state; Pedersen commitments and BLS signatures left discrete-log assumptions in the consensus path; the proof system's elliptic-curve foundation bound ledger soundness to Shor-vulnerable math; sharding was gestured at rather than specified; and per-transaction receipts could not survive a million TPS.

Attempt 4 (v3.0, this document): provable-first. The cryptographic commitment is the sole source of truth; the neural layer is a provable accelerator; the entire security-critical stack is hash- and lattice-based; sharding is a first-class subsystem with native cross-shard transactions; receipts are replaced by one

recursive batch proof, one compact attestation, and small inclusion witnesses; and the reward pool for "useful work" is right-sized to its advisory role, with the savings redirected to the people who built the network. Four attempts, one principle, finally stated without exceptions: prove the ledger; index it with mathematics; never confuse the two.

1.6 Design Principles

P0 — The commitment is the truth. The composite state commitment C_t is the sole authoritative representation of the ledger. Every validity proof binds to it; every derived structure (including e_t) is downstream of it and reconstructible from public data.

P1 — Post-quantum everywhere it matters. No elliptic-curve assumption, no pairing, no Pedersen algebra, no trusted setup appears in any security-critical role. Hash functions (BLAKE3), lattice signatures (ML-DSA), lattice KEMs (ML-KEM), lattice homomorphic encryption (NERV-Seal), and hash-based proof systems (STARKs) carry every load. The sole stated assumption beyond standard ones is Fiat–Shamir in the quantum random oracle model, which we state openly rather than inherit silently.

P2 — Authority/representation separation. As §1.3. Derived indexes may be lossy; commitments may not.

P3 — Identity frozen; knowledge alive. Canonical functions (W , the note model, the circuits, the update rules) are versioned and static between governance epochs. Learning is confined to advisory models whose failure is public and non-catastrophic.

P4 — Exogenous, sealed targets. Nothing is ever trained on its own output. Every learning signal is revealed reality that was cryptographically sealed (under the threshold channel) at prediction time.

P5 — Exactness in consensus. All consensus-relevant arithmetic is exact: integer and fixed-point operations with specified rounding, deterministic replay from genesis. No error bars, no tolerance windows, no votes on approximations.

P6 — Aggregate-only revelation. Individual plaintexts (deltas especially) are never decrypted by any single node. Threshold committees reveal sums; verifiable decryption makes the sums publicly auditable without any elliptic-curve commitment.

P7 — Out-of-sample rewards. Tokens are paid only for skill demonstrated on sealed targets after commitment — never for in-sample loss reduction.

P8 — Robust statistics throughout. Huber loss, clipped gradients, bounded blame; the network updates vigorously on ordinary events, cautiously on outliers, and never lets one loud event teach the lesson.

P9 — Honest scoping and honest claims. Privacy is claimed against named adversary models, not as a percentage. Throughput is claimed as an architecture target with published byte budgets, not as a benchmark. Post-quantum security is claimed as a construction, with its assumptions listed. A protocol that overclaims is a protocol that will be trusted once.

1.7 What Changed From v2.1, and Why (Review Response Register)

The v3.0 revision is a systematic response to the external technical review of v2.1. The register below maps each material finding to its resolution:

Review finding | v3.0 resolution

Circularity: proving a representation consistent with itself (items 1, 2, 25–27) | Commitment-first architecture; C_t is the sole authority; the proof pipeline of §2.2 mirrors the reviewer's recommended stack exactly

Fixed-dimensional embedding cannot bind an unbounded ledger (item 3) | Stated as a theorem of the design, not a defect hidden by it: e_t is a derived index with provable collisions; only C_t binds (§7.5)

$\delta \neq 0$ is not collision resistance (item 4) | Retained as a sanity check only;

injectivity-on-admissible-support retained as an index-completeness property; explicitly not a security claim (§7.4)

NWO must not secure funds (item 5) | The money layer never consults the neural layer (§3, §7)

Proof must bind to the current state commitment (item 6) | Every transaction proof includes a finalized NCT anchor within a bounded window, hashed into the proof's public inputs (§5.1)

Authenticated data structure required (item 7) | Note commitment tree + nullifier sparse Merkle tree + transit logs, all hash-based (§3)

Aggregation must not replace validity (item 8) | Per-transaction proofs precede aggregation; aggregation-soundness stated as a lemma (§5.6)

Affine term breaks additivity (item 9) | Zero bias in the canonical map, retained from v2.1 and restated with proof (§7.2)

"100% privacy" overclaims (items 10, 11, 29) | Property-wise privacy definitions with named adversary models (§2.6, §6.4)

Batch minimums for aggregation privacy (item 12) | $B_{\min}$ raised to 128 with roll-forward and optional committee padding (§6.4)

Homomorphic encryption underspecified (item 13) | NERV-Seal specified end to end: parameters, DKG, encryption, aggregation, verifiable threshold decryption, rotation, malicious-participant handling, noise budget (§6.3)

Ciphertext–delta binding required (item 14) | Well-formedness of the seal is proven inside each transaction's STARK; the aggregate opening is verifiable by anyone (§5.1, §6.3)

100k TPS needs end-to-end engineering; 1M needs parallelism (items 15, 16) | Bottleneck-by-bottleneck budget in §8.8; parallel ingress, proving, DA, and consensus throughout §8–§9

Sharding must be defined independently of the embedding (item 17) | Deterministic home-shard partition function; full cross-shard protocol (§8)

Dynamic sharding via ML is dangerous (item 18) | Deterministic hysteresis rules on finalized metrics; ML advisory only (§8.7)

Keep ML out of consensus (item 19) | Nothing canonical consults any model (§10)

Adam must not determine consensus state (item 20) | Forecaster is a deterministic integer-arithmetic replay of public data; committed as derived state, never as input (§10)

Entire stack must be PQ; BLS is a blocker (items 21, 22) | BLS removed; ML-DSA quorum certificates; full primitive-by-primitive PQ audit table (§9.4)

Proof-system assumptions must be examined (item 23) | All-STARK stack; no elliptic curves; QROM assumption stated (§5.7)

Per-transaction receipts do not scale (item 24) | One recursive batch proof, one compact batch-level attestation, ~600-byte inclusion witnesses (§11)

Formal specifications required (item 28) | Informal theorems A–H stated with their proof obligations (§13)

Claim wording (item 29) | Adopted verbatim in spirit (§2.6)

Tokenomics: NWO is only an optimization | Useful-work pool reduced to 3%; early contributors raised to 20% (§12)

2. Architecture Overview

2.1 The Four Layers

NERV is best understood as four stacked layers, each with its own primitives, guarantees, and failure semantics. The stack is deliberately aligned with the architecture the review recommended, because that architecture is correct:

Layer 1 – Custody (the cryptographic ledger of record). Private notes, BLAKE3 note commitments, nullifiers, the note commitment tree, the nullifier sparse Merkle tree, and the cross-shard transit logs. This layer owns money: who may spend what, and the prevention of double-spends. It is the sole source of truth. Nothing in this layer ever learns, adapts, or evolves; no elliptic curve, trusted setup, or hardware attestation appears in it. All of its security reduces to BLAKE3 and lattice signatures.

Layer 2 – Private transaction validity (the proof fabric). Transparent STARK proofs that each transaction is valid against an anchor of C_t : membership, ownership, nullifier authenticity, conservation, ranges, well-formed features, correct per-shard deltas, and well-formed sealed ciphertexts. Proofs aggregate through wallet → bundle → global-registry tiers, so that on-chain verification is $O(1)$ in throughput. This layer owns succinctness and privacy of attestation.

Layer 3 – Execution and acceleration (the neural overlay). The frozen public codec W , per-transaction and per-leg delta computation, the per-shard 512-byte embeddings, the forecasting substrate, and the consumers of forecasts (load management, cover-traffic shaping, anomaly flags). This layer owns expectations about the future and cheap public analytics. It learns every block; it is wrong in public; nothing canonical depends on it. If it vanished tomorrow, every coin would still be exactly where it is.

Layer 4 – Consensus and data availability. A post-quantum beacon chain and per-shard BFT committees, all signing with ML-DSA; hash-based erasure-coded data availability with sampling. Consensus finalizes the batch, the state commitment C_{t+1} , the aggregate transition, and the batch proof. The key rule, verbatim from the review and enforced by construction: consensus does not trust the neural overlay – it verifies a cryptographically proven relationship between the overlay's outputs and the authenticated ledger.

Privacy and transport cut across all four: the PQ-Sphinx mixnet (network anonymity), NERV-Seal (aggregate-only revelation of deltas), and the all-lattice signature and encryption stack.

2.2 The Canonical Pipeline

The lifeblood of the protocol is the following pipeline. Authority enters at the top and flows only downward through cryptographic bindings; the neural overlay sits to the side, fed by public data, feeding nothing back:

CRYPTOGRAPHIC LEDGER (Layer 1)

↓ $C_t = \text{Commit}(S_t)$ – the authenticated state

PRIVATE TRANSACTION (Layer 2)

↓ STARK validity proof, bound to a finalized anchor of C_t

VALID PER-SHARD DELTAS δ_i

↓ each δ_i proven, then sealed under its shard's threshold key

BATCH AGGREGATION

↓ $\Delta_B = \sum \delta_i$ — public linear operation on ciphertexts

NERV TRANSITION (Layer 3, derived)

↓ $e_{t+1} = e_t + \Delta_B$ — exact vector addition, advisory

CRYPTOGRAPHIC STATE UPDATE

↓ $C_{t+1} = \text{Update}(C_t, B)$ — the authoritative transition

RECURSIVE BATCH PROOF + BATCH ATTESTATION

↓ one proof, one compact attestation, per interval

POST-QUANTUM CONSENSUS (Layer 4)

↓ finality on C_{t+1} , Δ_B , and the proof

Two properties of this pipeline deserve emphasis. First, the neural transition and the cryptographic update are computed from the same proven deltas, in parallel, with no dependency between them; the embedding is a public mirror of the economy, not its master. Second, every arrow is a binding that survives quantum adversaries: hashes, lattices, or both — never discrete logs.

2.3 The Life of a Transaction (Bird's-Eye View)

A complete walkthrough with formal detail is Appendix A. In outline, for a transaction spending notes in two shards and creating outputs in two shards:

1. Construct legs — the wallet selects input notes, creates output notes, and assembles the transaction as a set of legs: one leg per shard it touches (each leg carries the nullifiers of inputs homed in that shard, the output commitments homed in that shard, the encrypted notes for that shard, and its fee share). One whole-transaction identity (txid) binds all legs.
2. Prove — the wallet produces a single transparent STARK covering the whole transaction: membership under an anchor, ownership, nullifier derivation, cross-shard conservation (inputs = outputs + fee over all legs combined), ranges, feature well-formedness, per-leg deltas, and well-formedness of each leg's sealed ciphertext. Wallet proving runs 0.5–2 seconds depending on device and is overlapped with mixnet transit, so user-perceived latency is approximately the mixnet's.
3. Seal — each leg's delta is encrypted under its shard's epoch threshold key (NERV-Seal, §6.3); the ciphertext and its hash join the leg.
4. Route — the transaction traverses five staked mixnet relays wrapped in post-quantum onion encryption with timing jitter and cover traffic; no observer links sender to transaction.
5. Bundle — untrusted aggregators collect transactions and fold their proofs into bundle STARKs (batched FRI verification amortizes the cost), submitted to the global registry with the transaction data.
6. Register — a rotating prover committee folds all bundles in a one-second interval into one global validity proof G_t , attesting "every transaction in the committed set T_t verified." T_t 's root is registered on the beacon chain. A transaction is valid protocol-wide the moment it enters T_t .

7. Include — each involved shard's next block includes the legs homed to it, referencing T_t by Merkle membership; the shard's state updates natively (note-tree inserts, nullifier inserts, transit entries) — plain hashing, no per-transaction ZK verification at all.
8. Forecast and seal — before decryption, every validator deterministically computes the forecaster's prediction of the shard's aggregate and commits its hash into the block header. The exam is locked.
9. Aggregate and reveal — the producer homomorphically sums the sealed deltas; the shard's decryption committee (t-of-n, with lattice zero-knowledge proofs of correct partial decryption) opens one aggregate Δ_B ; no individual delta is ever exposed.
10. Update and score — the shard embedding updates by vector addition; the committed forecast is scored against the revealed reality; the residual and the forecaster's deterministic update are recorded as a derived state.
11. Finalize — the shard committee signs the block with ML-DSA quorum certificates; the beacon chain finalizes shard headers. Single-shard finality $\approx 2-3$ seconds from registry inclusion.
12. Witness — the user receives a compact inclusion witness (~ 600 bytes): a Merkle path into the transaction tree, the block reference, and the finalized attestation chain. Cross-shard legs complete via the deterministic transit-mint rule within 2–4 additional seconds.

2.4 Reader's Map: Vocabulary

Term | Meaning

Private note | A secret record encoding ownership of an amount, spendable only by its holder

Note commitment | A sealed, hiding envelope $cm = \text{BLAKE3}(\text{"nerv.cm"} \parallel v \parallel p \parallel d \parallel r)$ binding a note without revealing it

Nullifier | A one-time public tag whose revelation marks a specific note as spent, without identifying it

State commitment C_t | The hash-based composite commitment to the entire authoritative state — the source of truth

Anchor | A finalized note-commitment-tree root, referenced by a transaction's proof, binding it to the authenticated state

Home shard | The deterministic shard assignment of an address and its notes: $\text{shard}(\text{addr}) = \text{BLAKE3}(\text{"nerv.shard"} \parallel \text{addr}) \bmod N$

Leg | The portion of a transaction (nullifiers, outputs, fees, sealed delta) homed to one shard

Transit log | The per-shard authenticated record of cross-shard value in escrow, enabling deterministic minting

Codec W | The public, frozen 64×256 matrix mapping transaction features to embedding deltas

State embedding e_t | The 512-byte, 64-dimensional public derived index of a shard's cumulative activity — a representation, never a commitment

NERV-Seal | The lattice-based linearly homomorphic threshold cryptosystem carrying sealed deltas (§6.3)

Aggregate-only revelation | The protocol rule that only sums of deltas, never individual deltas, are ever decrypted

Verifiable decryption | Threshold decryption in which every partial decryption carries a lattice zero-knowledge proof of correctness

STARK | A transparent, hash-based succinct zero-knowledge proof — no trusted setup, no elliptic curves

Bundle | 64–512 transactions folded into one aggregator STARK

Global validity registry (T_t, G_t) | The per-interval committed set of verified transactions and its recursive batch proof

ML-DSA / ML-KEM | NIST-standardized lattice signature and key-encapsulation primitives

Quorum certificate (QC) | A $2f+1$ -of- $3f+1$ committee attestation composed of ML-DSA signatures, hash-compressed in headers
Inclusion witness | The ~600-byte compact replacement for per-transaction receipts (§11)
Adam / Huber | The deterministic integer optimizer and bounded-blame loss that train the forecaster

2.5 Threat Model and Guarantees

Adversary | Sees | Guarantee NERV provides

Global passive network observer | Packet timings, sizes, IPs | PQ-Sphinx onion routing, exponential jitter ($\mu = 100$ ms, $\sigma = 200$ ms), and learned cover traffic; sender–transaction linkage probability quantified under a stated observer model (§6.4)

Mempool and DA observer | Public legs, nullifiers, commitments, sealed ciphertexts, fees | Shielded-pool cryptography; sealed delta channel; nothing links users, amounts, or counterparties; per-block aggregate only

Block producer | Legs, batch composition, sealed ciphertexts | Cannot read any individual delta; cannot reorder for profit (aggregation is order-invariant and inclusion order is canonicalized by fee-then-txid sort); inclusion steering is bounded and statistically visible in committed forecast residuals

$< \frac{1}{3}$ Byzantine validators | Their own views | BFT safety on C_t via ML-DSA quorum certificates; invalid blocks are natively checkable (all state updates are public hashing); rogue model updates impossible (the forecaster is a deterministic replay, not a vote)

Seal-committee collusion ($< t$ members) | Partial decryptions | Cannot open any aggregate or individual ciphertext; colluding $\geq t$ members can open individual ciphertexts, is detectable only ex post through governance forensics, and is bounded by slashing, stake, and epoch rotation (§6.3.7 – stated honestly)

Malicious prover / aggregator | Public transaction and proof data | Cannot forge validity (G_t must verify against BLAKE3-based FRI); redundant provers and an open challenge market make omission visible and slashable

DA adversary | Erasure-coded blobs | 2D Reed–Solomon sampling: data is available with overwhelming probability unless the adversary controls $\geq \frac{1}{3}$ of samples; all DA commitments are hash-based

Quantum adversary (future) | Everything recorded | All security-critical primitives are hash- or lattice-based at genesis; no BLS, no Pedersen, no elliptic-curve SNARK exists to break. The stated residual assumption is Fiat–Shamir in the QROM (§5.7)

Adversarial data (MEV spikes, spam cascades) | The aggregate stream | Huber loss caps per-dimension blame; gradient clipping caps update norms; anomaly flags are advisory; the ledger itself never moves – only expectations do

What NERV does not claim. The forecaster is adaptive statistics, not intelligence. Anomaly flags are advisory inputs to operations, not automatic slashing triggers. Delta-level privacy is computational and aggregate – a rank-64 projection mixed among ≥ 128 transactions – one tier below the note layer's privacy, and parameterized by W 's conditioning, which is re-reviewed at every governance epoch. Mixnet anonymity is claimed against an adversary observing less than a stated fraction of links and controlling less than a stated fraction of relays – not unconditionally. These limits are stated here because a protocol that overclaims is a protocol that will be trusted once.

2.6 Honest Claims

v3.0 adopts the review's discipline on claims, verbatim in spirit:

Privacy. NERV provides cryptographically enforced transaction confidentiality (amounts, parties, and note contents hidden from validators and observers), sender–receiver unlinkability at the data layer, and formally specified network-anonymity guarantees under a defined adversary model (§6.4). NERV does not claim "100% privacy." Privacy is claimed per property, per mechanism, per adversary.

Throughput. NERV's architecture is designed for horizontal scaling toward 1,000,000 transactions per second across up to 1,024 shards, with per-shard sustained capacity of 1–10k TPS and an honest byte-level budget for every component (§8.8). Genesis deployments target 10^4 – 10^5 TPS aggregate. No throughput number in this document is a benchmark; each is a design target with its cost stated.

Post-quantum security. NERV is post-quantum by construction: every security-critical primitive — state commitments, signatures, quorum certificates, transport encryption, note encryption, the sealed delta channel, and the entire proof stack — is hash-based or lattice-based, with parameters targeting ≥ 128 -bit post-quantum security (96-bit for the advisory-tier seal, whose role is bounded and stated). The single assumption beyond standard ones — Fiat–Shamir in the quantum random oracle model for STARK soundness — is named rather than inherited. A complete primitive-by-primitive audit appears in §9.4.

Self-evolution. The network's forecasting layer demonstrably improves out-of-sample on cryptographically sealed targets, in public, for a deliberately modest reward pool. It is a bonus feature of the protocol, not a pillar of its security, and it is funded accordingly (§12).

Section 3

3. The Accounting Model: Notes, Commitments, Nullifiers, Legs

3.1 Design Intent: Authority Is Boring

NERV's custody layer is conventional on purpose. Before any neural machinery appears, the question "who owns what, and how do we prevent double-spends?" must be answered by the most battle-tested construction in shielded finance: the note/nullifier model pioneered by Zcash and audited by a decade of adversarial production use. The external review of v2.1 confirmed what the design lineage had already taught: conventionality here is not a lack of ambition — it is the load-bearing conservatism that lets everything unusual live safely downstream.

Principle P0 applies most strictly in this layer: the commitment is the truth. Nothing in the custody layer ever learns, adapts, or evolves. Its functions are fixed at genesis, its arithmetic is integer-exact, and every one of its guarantees reduces to BLAKE3, ML-DSA, ML-KEM, or module-LWE — hash and lattice assumptions only.

The reader should hold one framing throughout, corrected from v2.1: NERV's ledger is two parallel structures. The first — this section — is the authoritative ledger: it tracks ownership exactly, prevents double-spends, and enforces conservation of value, and its binding object is the composite state commitment C_t (Section 4). The second — Section 7 — is a derived public index: a 512-byte lossy fingerprint of economic activity. In v2.1 the second structure sat inside the state root and occupied its semantic center. In v3.0 it is committed as derived data and is never consulted by anything authoritative. Money lives in cryptography; only summaries live in linear algebra; and in v3.0, the summaries are provably downstream.

3.2 Private Notes and the Key Hierarchy

A private note is the atomic unit of ownership: a secret record that binds an amount to a spending authority. The note plaintext is never published. Its fields:

Field | Description

v | Value in nano-NERV (integer, $0 < v \leq 2^{60}$; the cap reserves guard bits for arithmetic)

p | Unique note nonce (256-bit random, chosen by the sender at creation)

d | Recipient's diversified delivery public key (one of many one-time addresses per wallet)

memo | Optional encrypted memo (up to 80 bytes)

Address and key hierarchy. A wallet derives everything from a 256-bit master seed (BIP-39 compatible, PBKDF2-HMAC-SHA512): a spending key `sk_spend`, a note-derivation key `nk`, and detection keys, via domain-separated BLAKE3-KDF expansion. Recipients publish diversified addresses — cheap, unlimited, unlinkable delivery points — so each note's delivery key `d` can be unique. A viewing/detection key can be delegated for regulated wallets without ever exposing spending capability: detection keys reveal which notes are yours; only `sk_spend` can move them.

Shard-aware addresses. Every address is deterministically homed to a shard:

$$\text{shard}(\text{addr}) = \text{BLAKE3}(\text{"nerv.shard"} \parallel \text{addr}) \bmod N_s$$

where N_s is the active shard count (a governance parameter; 64 at genesis, designed to grow to 1,024 — Section 8). Addresses are free to mint, so wallets naturally diversify across shards. Because notes are homed by delivery address, a wallet scans only the encrypted-note streams of the shards where its addresses live — a privacy and scalability synergy: scanning traffic is shard-local, and no observer of a shard learns anything about wallets homed elsewhere. Address validity includes the shard tag, so cross-shard confusion is a type error, not an attack surface.

Note encryption. The note plaintext is sealed to the recipient using hybrid PQ encryption:

- $(ss, ct) = \text{ML-KEM-768.Encapsulate}(d)$ — one encapsulation per recipient per transaction
- $k = \text{BLAKE3-KDF}(ss, \text{"nerv.note"})$
- $\text{enc_note} = \text{ChaCha20-Poly1305.Encrypt}(k, \text{note_plaintext})$

The encrypted note is published on-chain as an opaque blob. Recipients discover incoming notes by trial-decrypting new blobs with their detection keys — standard, private, and requiring no interaction with the sender or any server.

3.3 Note Commitments and the Note Commitment Tree

To spend a note later, the spender must prove it exists — without revealing which note. Each note is therefore published only as a commitment:

$$cm = \text{BLAKE3}(\text{"nerv.cm"} \parallel v \parallel p \parallel d \parallel r)$$

where r is a fresh 256-bit blinding factor chosen by the sender. This construction is hiding (with r uniform, cm is computationally indistinguishable from random under the standard hash heuristic — and see §3.5

for why this tier of hiding is exactly sufficient) and binding (finding two distinct note contents with the same cm is a BLAKE3 collision — infeasible for classical and quantum adversaries alike, since Grover only halves the effective security of preimage-style attacks and collision search on 256-bit outputs remains far beyond practical quantum capability; the full audit is §9.4).

Each shard maintains its own append-only note commitment tree (NCT): a BLAKE3 binary Merkle tree of depth 32 (capacity ≈ 4.3 billion notes per shard; $\approx 4.4 \times 10^{12}$ across 1,024 shards), with its root committed in every shard header. Batched insertion is standard: each block's k output commitments form a subtree appended in canonical order, costing $O(k \log k)$ hashes. Membership in the shard's NCT is what makes a note spendable; the membership proof is revealed only inside a zero-knowledge circuit (Section 5), never in public.

3.4 Nullifiers, the Spent Set, and the Transit Log

Spending must be public in effect (a note can never be spent twice) yet private in identity (no one learns which note was spent). The instrument is the nullifier:

$$nf = \text{BLAKE3-ToField}("nerv.nf" \parallel nk \parallel p)$$

Derived from the spender's secret note key and the note's unique nonce, the nullifier is a one-time pseudorandom tag. It cannot be linked back to cm (both p and nk are hidden — one by the commitment's blinding, the other by secrecy). The protocol maintains, per shard, a public nullifier set: a BLAKE3 sparse Merkle tree of depth 256 over 256-bit keys (empty-default subtrees, standard caching; batch updates cost $O(k \cdot 256)$ hashes — a few milliseconds per 10,000 nullifiers on commodity hardware). Non-membership proofs (a path to a default leaf) exist for light clients and fraud proofs, but full nodes enforce the rule directly:

A leg is invalid at settlement if any of its nullifiers already appears in the shard's nullifier set, or appears earlier in the same block under canonical ordering; on settlement, they are inserted.

Why a per-shard nullifier set suffices — a first-principles point worth stating precisely. Notes are homed at creation: a note exists in exactly one shard's NCT, determined by its delivery address. Its nullifier is therefore checked in exactly one shard. The double-spend of a given note is a single-shard predicate by construction; there is no cross-shard double-spend window, no global nullifier set, and no cross-shard locking. Sharding does not weaken double-spend prevention; it partitions it.

The transit log. Each shard additionally maintains a settled-leg index: a BLAKE3-authenticated sorted map keyed by $H(\text{txid} \parallel \text{shard} \parallel \text{leg_index})$, with membership proofs of $O(\log n)$ and size ≈ 1 KB. It serves two purposes: replay prevention for legs (each leg settles at most once per shard), and — as its name advertises — it is the record other shards consult to complete cross-shard transactions (§4.5). Value never "escrows" in the transit log; only completion conditions do. The log's root is committed in every shard header.

3.5 The Commitment Inventory: Why Pedersen Is Gone

v2.1 used Pedersen vector commitments $C = \text{PedVec}(\delta; r)$ to bind each sealed delta for public aggregate audit, bounding their role to auditability on the theory that a quantum adversary who forged them would

corrupt only audits. The v3.0 revision removes Pedersen entirely, and the reasoning is worth doing from first principles, because it generalizes.

What each commitment must provide. A commitment scheme offers hiding (the committed value is concealed) and binding (the committer cannot later open to a different value). Different call sites in a blockchain need different extras: additive homomorphism (to sum public commitments), post-quantum binding (to survive Shor), cheap in-circuit verification (to prove statements about commitments inside a STARK), threshold opening (to reveal an aggregate without revealing parts), and size. No single classical scheme provides all of these; the design question is which object serves each call site with the weakest sufficient assumptions.

The composite-wall principle. A privacy wall is as strong as its weakest layer. If a secret travels through a channel protected at tier X and is also committed under a scheme with perfect hiding, the adversary's view is still bounded by tier $X - \min(\text{perfect}, X) = X$. This principle, applied honestly, dissolves most of the nostalgia for Pedersen:

- v2.1's delta wall was Pedersen (perfectly hiding) + a module-LWE threshold channel + a committee. Its effective privacy was already capped at module-LWE. Replacing the perfectly-hiding layer with an LWE-hiding layer does not lower the wall: $\min(\text{LWE}, \text{LWE}) = \text{LWE}$. What the replacement does remove is a discrete-log binding assumption — an assumption that falls to Shor — from the audit path. v2.1 kept a quantum-forgable object guarding the very audits meant to detect quantum-era misbehavior. That was the unsoundness; the fix is not a bigger role for Pedersen but no role at all.
- The same principle justifies hash-based note commitments: the note plaintext is already sealed under ML-KEM-768 and ChaCha20-Poly1305, so the commitment's hiding needs only to exceed that tier — which BLAKE3 with 256-bit blinding does comfortably — while its binding (collision resistance) is fully post-quantum, which Pedersen's never was.

The candidate families, honestly compared:

Family	Hiding	Binding	Homomorphic	PQ	In-circuit cost	Size
Pedersen (EC)	Perfect	DLOG — broken by Shor	Additive	No	MSM, ~11K constraints (v2.1)	48 B
Hash commitment (BLAKE3)	Computational (256-bit blinding)	Collision resistance	No	Yes	Native hash	32 B
Ajtai / module-SIS commitment ($C = A \cdot [m; r] \bmod q$)	LWE	SIS	Additive	Yes	Mod-q linear algebra, lookup-friendly	~1–3 KB
Verifiable encryption as commitment (NERV-Seal)	LWE	STARK soundness (QROM) on the encryption statement	Additive by scheme linearity	Yes	Mod-q linear algebra + smallness checks	The ciphertext itself (KB-scale)

The chosen assignment — one object per job, no object with a Shor-vulnerable assumption:

- Custody (note commitments): BLAKE3 hash commitments. Binding — the property that matters most where money lives — is post-quantum by construction; hiding exceeds the tier of the note-encryption wall around it; in-circuit verification is a native hash in a hash-native proof system.
- The delta channel: the seal is the commitment. v2.1 needed two objects — a Pedersen point per transaction, plus a threshold ciphertext — because the commitment's binding had to be checked against

something the network could open. v3.0 collapses them. Each transaction's STARK proves, as one of its ten statements, that its sealed ciphertext is a well-formed encryption of exactly the proven delta with bounded noise (verifiable encryption). The ciphertext is therefore publicly binding (forging a mismatched pair requires breaking the STARK), privately hiding (module-LWE), and additively homomorphic (the scheme's linearity). The aggregate audit needs no separate commitment at all: anyone can compute the batch ciphertext $ct_B = \sum ct_i$ from public leg data; the decryption committee opens ct_B with verifiable partial decryptions; the public verifies both the sum and the opening. One object does transport, binding, and audit. The marginal cost of dropping Pedersen is negative — the protocol deletes an object, 48 bytes per transaction, and roughly 11,000 circuit constraints.

- Standalone Ajtai commitments are specified as the standard algebra inside NERV-Seal (its public matrix and linear structure are exactly Ajtai's), so if governance ever requires detached homomorphic commitments — a shielded-but-auditable value pool, for instance — the same algebra serves with no new assumptions.
- Proof-system commitments are FRI — hash-based, transparent, post-quantum — rather than KZG (pairings) or IPA (discrete log). This is why the entire proof stack is STARK-family (Section 5).
- The state commitment is BLAKE3 (Section 4) — representation-independent, unbounded-capacity, collision-binding.

The complete inventory, as audited in §9.4:

Object	Binds / protects	Hiding	Binding	Assumption	Size
Note commitment cm	Note contents to NCT position	Computational (256-bit r)	Collision resistance	BLAKE3	32 B
Nullifier nf	Spent-ness to secret key + nonce	— (pseudorandom tag)	Preimage resistance	BLAKE3	32 B
NCT / nullifier tree / transit log roots	Membership and freshness	—	Collision resistance	BLAKE3	32 B each
Composite state commitment C_t	The entire authoritative state	—	Collision resistance	BLAKE3	32 B
Batch seal ct_B	The aggregate delta Δ_B	Module-LWE	STARK soundness + LWE	module-LWE / QROM	~KB (per batch)
Polynomial commitments (in proofs)	Prover's claimed polynomials	—	FRI binding	BLAKE3 / QROM	In-proof

What v3.0 gives up relative to Pedersen, stated plainly: perfect hiding of the delta binding (now computational, LWE-tier, 96-bit parameters — coherent because the whole wall was already LWE-tier) and 48-byte compactness of a per-transaction point (irrelevant, because the point is gone and the ciphertext was required anyway). What it gains: the complete elimination of elliptic-curve assumptions from every commitment and every audit path in the protocol.

3.6 The Transaction: Legs, Shells, and Sealed Deltas

In a sharded ledger, the transaction itself must be sharded. NERV makes multi-shard transactions first-class: a single transaction, a single proof, atomic semantics, settlement spanning shards.

Structure. A transaction is a set of legs — one per shard it touches:

txid = BLAKE3(canonical serialization of all legs)

Each leg is the portion of the transaction homed to one shard and contains:

Public shell (cleartext) | Sealed (threshold-encrypted)
Nullifiers of input notes homed to this shard | The leg's 64-dimensional delta δ_{leg}
Output note commitments homed to this shard | (encrypted under this shard's epoch key)
Encrypted output notes for recipients homed here
Declared fee share (integer nano-NERV)
Anchor (an NCT root of this shard, within the freshness window)
Weight-version tag (which frozen W computed the delta)
Expiry height
The leg's sealed ciphertext ct_{leg}

The whole-transaction proof (one STARK, Section 5) covers all legs jointly: membership of every input under its own shard's anchor, ownership, nullifier derivation, ranges — and conservation globally: $\Sigma \text{ inputs} = \Sigma \text{ outputs} + \Sigma \text{ fees}$ across all legs combined. Value may enter a transaction in shard A and leave it in shards B and C; the proof enforces that the books balance across the union.

Feature vectors are per leg. Each leg carries its own feature vector ΔS_{leg} (account slots for the addresses homed in that shard, plus this leg's volume, fee, count, and log rails) and its own delta $\delta_{\text{leg}} = W \cdot \Delta S_{\text{leg}}$, sealed under that shard's epoch threshold key. Each shard's embedding therefore accumulates exactly the activity that settled in that shard — the index remains shard-local, which is what makes the per-shard embedding meaningful and the forecaster's job tractable. The feature layout is specified in Section 7.

The common case is single-shard: a payment where sender and recipient addresses share a shard is one leg, settles in one block, and touches nothing else. The interesting case is cross-shard:

Worked example. Alice (addresses homed to shard 7) pays Bob (address homed to shard 40) and Carol (address homed to shard 7). The transaction has two legs. Leg 7: nullifiers of Alice's input notes, output commitments for Carol's note and Alice's change, Carol's and Alice's encrypted notes, fee share, sealed δ_7 . Leg 40: output commitment and encrypted note for Bob, fee share, sealed δ_{40} . One STARK proves the whole transaction valid against anchors of shards 7 and 40. Shard 7 settles leg 7 unconditionally (its inputs live there). Shard 40 settles leg 40 as soon as leg 7 is finalized — which shard 40's validators verify deterministically from beacon data (§4.5). Bob's note exists in shard 40's NCT a couple of finality epochs after Alice's inputs were consumed in shard 7. No lock, no coordinator, no trusted relay.

Sizes (the honest budget; full accounting in §8.8). A typical 2-input, 2-output single-shard leg: shell ≈ 700 B; encrypted notes ≈ 1.3 KB (dominated by one ML-KEM-768 encapsulation per recipient); sealed delta $\approx 4\text{--}5$ KB (NERV-Seal parameters, §6.3 — the largest per-transaction object, explicitly targeted by the compression roadmap, §14); proof cost amortized to $\approx 200\text{--}400$ B per transaction by bundle aggregation (§5.5).

3.7 What the Whole-Transaction Proof Must Establish

The complete statement list is the province of Section 5; the ledger-level contract it must satisfy is short enough to state here, because it is the review's item 6 made exact. For every transaction, the proof establishes, in zero knowledge:

1. Every input note commitment exists in its shard's NCT, under the anchored root — the spent value exists in the authenticated state.
2. The spender knows the spending authority for those notes.
3. Every published nullifier is correctly derived from the spent note's secret key and nonce.
4. $\Sigma \text{ inputs} = \Sigma \text{ outputs} + \Sigma \text{ fees}$, over all legs combined — value is conserved globally.
5. Every value lies in $[0, 2^{60}]$.
6. Every leg's feature vector is well-formed and on the admissible support (§7.4).
7. Every leg's delta is exactly $W_{\tau} \cdot \Delta S_{\text{leg}}$ under the tagged frozen encoder.
8. Every leg's sealed ciphertext is a well-formed encryption of exactly that delta, with bounded noise (verifiable encryption — the binding of §3.5).

Conservation is enforced mathematically per transaction — not statistically, not by aggregate audit, not by machine learning. This sentence survives from v2.1 unchanged because it was true there and remains true here; what changed is everything around it.

3.8 Genesis and the Shielded Supply

The entire supply (10,000,000,000 NERV; allocation in Section 12) is created at genesis as an initial distribution of private notes committed into the genesis NCTs of their home shards, per the published allocation. The genesis state C_0 commits this distribution, the initial parameters, and nothing else. No inflation follows: the fee market and the Useful-Work pool (paid from genesis reserves, not emission) are the only compensation sources. Shielded→transparent burns are always available (a note spent with no output note and a public burn commitment, verifiable by anyone); transparent→shielded minting windows are governance-gated.

3.9 What This Layer Deliberately Does Not Do

- It does not learn, adapt, or reference any model. Total corruption of Sections 7 and 10 cannot mint, steal, unlock, or reorder a single note. Delete the embedding from every header and the ledger remains fully verifiable from C_0 .
- It hosts no general smart contracts at genesis. NERV v3.0 is a payments network; the note model's extension points are a governance-scoped roadmap item.
- It decrypts nothing. Every plaintext that must exist for a user is produced by that user's wallet; every value the network needs is either public in the shell or revealed only as a per-shard aggregate (Section 6).
- It escrows no value across shards. The transit log carries completion conditions, not coins; in-flight value remains cryptographically conserved at every instant (§4.5). No bridge, no wrapped assets, no trusted third shard.

Section 4

4. The State Commitment: A Mathematically Strong Layer 1

4.1 First Principles: From Representation to Commitment

The review of v2.1 identified, correctly, the single question on which the whole architecture turns: does the chain prove that a transaction is valid with respect to an independently authenticated ledger state — or merely that an output is consistent with a representation generated by the same transformation being validated? The first is a ledger. The second is a mirror describing itself.

The answer in v3.0 is architectural, not rhetorical. Four properties define a ledger commitment, and each is mapped to a mechanism rather than a hope:

Property | Mechanism in v3.0

Binding — infeasible to produce two materially different states with the same commitment | C_t is a BLAKE3 hash over every authoritative component; distinct states require a collision

Authenticity — a state transition is tied to the previously authenticated state | Hash-chained headers; every block commits its predecessor's C_t ; ML-DSA quorum certificates; beacon finality

Soundness — an invalid transaction cannot produce an accepted state commitment | Deterministic execution (§4.3): settlement requires registry proof validity, nullifier freshness, and exact root recomputation — all public checks

Persistence — a finalized state cannot change without detection | Beacon finality: no reorganization past a finalized interval without forging ML-DSA quorum certificates

The linearity of W provides none of these properties — and none of them is asked of it. The canonical flow, which every later section elaborates and never violates:

$C_t = \text{Commit}(S_t)$ — the authenticated state, the sole source of truth

↓

each transaction proves validity against an anchor of C_t

↓

settled legs update the authoritative structures

↓

$C_{t+1} = \text{Update}(C_t, B)$ — exact, deterministic, publicly recomputable

↓

$e_{t+1} = e_t + \Delta_B$ — derived, advisory, committed as derived data only

The system proves "the transaction is valid with respect to an independently authenticated ledger state." It never proves "the transformation agrees with itself." That is the whole of item 25, resolved by construction.

4.2 The Shard State S_t and the Composite Commitment C_t

The authoritative state of shard s at height h is the tuple:

$S_t = (nct_root_t, nullifier_root_t, transit_root_t, params_root_t, prev, height)$

and the block commits the composite root:

$C_t = \text{BLAKE3}("nerv.state" \parallel nct_root_t \parallel nullifier_root_t \parallel transit_root_t \parallel params_root_t \parallel prev \parallel height)$

exactly 32 bytes, reconciling validators, light clients, and inclusion witnesses against a single hash of everything canonical. Note what is absent: the embedding. The advisory state is committed separately as the derived-state root:

$$D_t = \text{BLAKE3}(\text{"nerv.derived"} \parallel e_t \parallel \text{forecaster_state_root}_t)$$

which appears in the shard header for deterministic replay and public audit of the knowledge layer, but is not part of C_t . The separation is deliberate and load-bearing: D_t is a pure function of public history (revealed aggregates), recomputable by anyone; if two implementations ever disagreed about D_t , the disagreement would be a public, slashable determinism bug in the advisory layer — and custody would be untouched, because no validity rule references D_t . This is the state-level resolution of the review's warning that an Adam-updated model must never determine consensus state: the model is replayed, not trusted; and even its replay is outside the authority of C_t .

Hash output length. All commitments are 256-bit. Quantum collision search on a 256-bit hash is bounded by the Brassard–Høyer–Tapp algorithm at $\approx 2^{85}$ operations with correspondingly astronomical quantum memory requirements — universally assessed as beyond any plausible adversary for the epochs this protocol must survive. The full accounting appears in §9.4; governance retains the option of widening the composite commitment at a coordinated upgrade should the consensus of cryptanalysis ever shift.

4.3 The Exact Update Function

A shard block body is an ordered list of settled legs plus the producer's bookkeeping. The ordering rule is deterministic: after validity filtering, legs are canonically ordered by $(\text{txid}, \text{leg_index})$. The producer's discretion is confined to which legs to include — never their order — which forecloses reordering games at the data layer and makes nullifier-conflict resolution within a block deterministic.

Execution is a pure function of $(\text{state}, \text{block})$. For each leg, in canonical order:

1. Registry validity: the leg carries a Merkle membership witness that $\text{txid} \in T_\tau$, the transaction set of a finalized registry interval τ , whose root and recursive proof G_τ are finalized on the beacon. The witness is a few hundred bytes; the proof G_τ is verified once per interval, never per transaction.
2. Anchor freshness: the leg's anchor is an NCT root from one of the last $A = 64$ finalized headers of this shard (a window measured in minutes — generous against mixnet transit and proving time, short enough to bind the proof to recent authenticated state).
3. Nullifier freshness: each of the leg's nullifiers is absent from the nullifier set and from earlier insertions in this block.
4. Transit freshness: $H(\text{txid} \parallel \text{shard} \parallel \text{leg})$ is absent from the transit log (no leg settles twice).
5. Issue-condition (cross-shard legs only): if the leg creates outputs but consumes no inputs in this shard, every sibling spend leg of the same txid is finalized in its home shard (§4.5).
6. Application: nullifiers are inserted into the nullifier tree; output commitments are appended to the NCT in canonical order; the transit-log key is inserted; the leg's ciphertext is added into the batch sum $\text{ct}_B = \sum \text{ct}_{\text{leg}} \pmod q$, per NERV-Seal).

The header then commits: prev, height, the recomputed component roots, D_t , the hash of ct_B , the revealed aggregate Δ_B of the previous block (whose opening ceremony completed before this header),

the registry reference (T_{τ} root, interval index), the public fee total, the producer's payout address, and the hash of the block's quorum certificate. $C_{\{t+1\}}$ is the composite hash of the tuple.

Every step is public hashing, public set-membership, and public modular addition. A full node validates a block without performing a single zero-knowledge verification — the only cryptographic proof verified on the consensus path is G_{τ} , once per second, on the beacon. The chain's verification load is therefore $O(1)$ in throughput, which is the property that keeps light clients under 100 KB of headers forever and makes one million transactions per second a networking problem rather than a verification problem.

4.4 Anchors and the Two-Phase Validity Model

v3.0 makes transaction validity a two-phase notion, specified precisely because the review demanded precision on what a proof is actually bound to:

Phase 1 — registry validity (global, once). The whole-transaction STARK is verified when the transaction enters the registry set T_{τ} . The proof's public inputs include the anchors it references — one NCT root per input shard — and these anchors are, by rule 2 above, roots that appeared in finalized headers, i.e., roots that are inside the hash chain of C_t . A transaction cannot be proven "valid against a fabricated or historical-floating state": the state it is valid against is identified, within a bounded window, by finalized commitments.

Phase 2 — settlement (per shard, once). Freshness — that the nullifiers have not been spent since the anchor — is enforced at settlement time by the shard itself, against its current nullifier set. Two conflicting transactions can both be registry-valid (both proved honestly against a common anchor); only one can settle. The loser is simply void: its nullifiers were never inserted, its inputs remain spendable, its outputs never existed. This is standard UTXO race semantics — the same semantics every note-based chain has — with the race confined, by homing, to a single shard per note.

The binding the review asked for is therefore exact and two-sided: the proof binds to authenticated past state (anchors inside C_t); the settlement binds to current state (freshness inside C_t). Neither is a floating assertion.

4.5 Cross-Shard Settlement: The Transit Rule

Cross-shard atomicity in NERV is a construction, not a coordination protocol. The rule:

A leg that consumes inputs (a spend leg) settles unconditionally, exactly like a single-shard transaction. A leg that only creates outputs (an issue leg) settles only once every sibling spend leg of the same txid is finalized in its home shard.

Verification of the condition is deterministic. Each shard's validators track the beacon chain, which finalizes every shard's headers; a sibling spend leg's settlement is publicly provable by a membership witness into that shard's finalized transit log (a ≈ 1 KB sorted-map proof against a root the beacon finalized). Because all honest validators see the same beacon, they evaluate the same condition and reach the same verdict — no cross-shard voting on facts, no asynchronous bug class of "I thought you had finalized."

Five consequences, each stated as a small lemma with its proof obligation listed in §13:

Atomicity (T5). A transaction either completes — every leg eventually settles — or fails cleanly: if any spend leg never settles (censorship or expiry), all its issue legs are permanently void and the inputs were never consumed; if all spend legs settle, every issue leg is permissionlessly completable, forever. There is no reachable state in which value is created without corresponding value destroyed. The "escrow" during transit is not a custodian: it is the deterministic rule itself, and the in-flight value is cryptographically conserved at every instant — global supply equals genesis supply minus burns minus in-flight, provable from public data at all times.

Deadlock-freedom. The dependency graph is bipartite and one-directional: issue legs depend only on spend legs; spend legs depend on nothing. There are no cycles; once data is available, any transaction completes within two finality epochs.

Liveness. Issue legs are public data in the registry. Anyone — the recipient's wallet, a paid relayer, an altruist — may submit an issue leg for settlement. Recipients are structurally incentivized to complete their own payments.

Censorship honesty. A censored spend leg expires; the transaction voids; inputs are intact. Nothing is stolen; the failure is visible and attributable to the censoring shard's producer set (and priced by its inclusion market).

Abandonment honesty. An issue leg that is never completed by anyone is a provable burn — the value leaves circulation, publicly and auditably. Wallet software completes legs automatically; this failure mode exists, is stated, and is the user's own.

4.6 Fees, Producers, and Header Discipline

Fees are public integers declared per leg and accrue to the producer of the shard that settles each leg — both shards of a cross-shard payment are paid for their work. Accrual is transparent: the block header records the fee total and the producer's payout address, and producer balances are explicit header-chain records, spendable through the standard governance-gated mint-to-shielded window. There are no hidden fee flows, and a producer cannot reorder for profit (canonical leg order) or inflate fees (public arithmetic).

Quorum certificates are hash-compressed: the header carries QC_hash, and the full ML-DSA certificate — signatures of a committee whose members are sampled per epoch (Section 8) — resides in the block body. The beacon folds an entire interval's shard certificates into a single interval attestation signed by the beacon committee, amortizing post-quantum signature bulk (ML-DSA signatures are kilobyte-scale, not the 96 bytes of BLS — the price of removing pairings, paid once per second network-wide, not once per transaction). The full attestation design, and its synergy with the receipt reform of Section 11, is specified there.

4.7 The Beacon and the Global Commitment

The beacon chain is deliberately small and boring: a modest, rotating validator set; one-second intervals; three jobs. It finalizes shard headers, committing the global root G_t — a Merkle root over all shard headers — which is the single object a light client verifies to anchor its entire worldview. It hosts the global validity registry (T_t , G_t). And it issues the interval attestations. Cross-shard conditions, light clients, and inclusion witnesses all reduce to: one beacon verification, then hashes. Consensus mechanics, committee sampling, and the liveness analysis are Section 8's subject; Layer 1's requirement of the

beacon is only what is stated here — it is the common clock and the common notary that make cross-shard determinism possible.

4.8 Layer-1 Soundness: Informal Theorems

The following statements are the load-bearing claims of Layer 1. Each is stated informally here, with its full formulation and proof obligations collected in Section 13.

T1 — Binding. C_t is a binding commitment to S_t : producing $S \neq S'$ with $\text{Commit}(S) = \text{Commit}(S')$ requires a BLAKE3 collision. (Classical: 2^{128} ; quantum: $\geq 2^{85}$ under memory-discounted collision search.)

T2 — State soundness. For any finalized shard chain, every $C_{\{t+1\}}$ equals the exact deterministic application of Update to (C_t, B_t) ; every settled leg's transaction was registry-valid at a finalized interval (STARK-verified under G_τ , Fiat-Shamir in the QROM); and every settled leg was fresh at settlement. Deviation requires forging ML-DSA, breaking BLAKE3, or breaking STARK soundness.

T3 — Conservation. For every settled transaction, $\Sigma \text{ inputs} = \Sigma \text{ outputs} + \text{fees}$, proven inside its STARK; therefore, at every finalized height, the total note value in all NCTs equals genesis supply minus burned and in-flight value — a public, checkable identity.

T4 — Persistence. Past beacon finality, headers cannot change without forging ML-DSA quorum certificates from $\geq \frac{2}{3}$ of a committee.

T5 — Cross-shard atomicity. As stated in §4.5: no reachable state creates value without destroying equal value; completion is deadlock-free and permissionless.

The pattern across all five is the pattern of the whole revision: every guarantee rests on hashes, lattices, or transparent proof-system soundness — never on a learned function, never on a fixed-dimensional projection, never on silicon. The embedding of Section 7 will be described, deliberately, as an object that could be deleted without invalidating a single theorem above.

The next section builds the machinery that makes T2 cheap: the proof fabric — transparent STARK circuits, the three-tier aggregation pipeline from wallet to global registry, and the one-verification-per-second property that lets consensus scale horizontally with shards rather than vertically with proofs.

Section 5

5. The Proof Fabric: LatentLedger Transparent

5.1 What a Transaction Must Prove

Every NERV transaction arrives with one transparent zero-knowledge proof — a STARK — that convinces any verifier of the following statements while revealing nothing beyond the public shell. The list is v2.1's list, corrected in two places (the Pedersen statement is gone; the sealing statement is strengthened from a consistency check to full verifiable encryption), and extended for sharding:

| Statement proven | Why it matters

- 1 | Each input note commitment exists in its home shard's note commitment tree, under the anchored root | The spent value exists in the authenticated state (review item 6)
- 2 | The spender knows the spending authority for those notes | Only owners can spend
- 3 | Each published nullifier is correctly derived from the spent note's secret key and nonce | Double-spend tags are authentic
- 4 | $\Sigma \text{ inputs} = \Sigma \text{ outputs} + \Sigma \text{ fees}$, over all legs of all shards combined | Value is conserved globally, across shard boundaries
- 5 | Every value lies in $[0, 2^{60}]$ | No overflow, no negative money
- 6 | Every leg's feature vector is well-formed and on the admissible support (≤ 6 active account slots per leg; correct volume, fee, count, and log rails) | The encoder is fed honest inputs
- 7 | Every leg's delta is exactly $\delta_{\text{leg}} = W_{\tau} \cdot \Delta S_{\text{leg}}$ under the tagged, frozen encoder version | The delta is correct
- 8 | Every leg's $\delta_{\text{leg}} \neq 0$ | Every provable transaction moves the public index (a sanity check — explicitly not collision resistance; §7.3)
- 9 | The seal's plaintext digits encode exactly δ_{leg} , in the canonical digit decomposition | The ciphertext and the proven delta refer to the same value (review item 14)
- 10 | The seal ciphertext is a well-formed NERV-Seal encryption under the shard's epoch key, with in-circuit-bounded small randomness and noise | No grieving via malformed ciphertexts; the binding of §3.5
- 11 | The proof's Fiat–Shamir transcript includes the binding challenge $H(\text{all nullifiers} \parallel \text{txid} \parallel \text{shell digest})$ | Replays and proof transplants break verification

The proof's public inputs are the shell digest, one NCT anchor per input shard (each within the freshness window of finalized headers — §4.4), the seal epoch key identifier, the encoder version tag, and the expiry height. This is the review's required binding made literal: a transaction cannot be proven valid against a floating, historical, or fabricated state, because the state it is valid against is identified by roots inside the hash chain of C_t .

5.2 One Proof, Two Modules

v2.1 split its bundle into two sub-proofs ($\pi_{\text{spend}}, \pi_{\delta}$) connected by a shared feature digest. v3.0 retains the seam at the circuit level, not the proof level: the transaction STARK contains two modules — the custody module (statements 1–5) and the delta module (statements 6–10) — sharing one Fiat–Shamir transcript and one binding challenge.

The reasons for the change are practical. Two proofs per transaction would double the aggregation load of §5.5, and STARK proof sizes make per-transaction objects expensive enough already. But the original rationale for the split survives intact, through versioned verification: the proof system's verification key is tagged with a circuit version, and the registry accepts any version whose key is registered on the beacon. A governance upgrade to the delta module (a new encoder feature layout, a new digit encoding) deploys as a new version with an overlapping acceptance window, while the custody module's version changes approximately never. The modules remain independently auditable, benchmarkable, and replaceable; the wallet emits one object.

5.3 The AIR and the Chips: A Dual-Hash Discipline

The proof system is an AIR (algebraic intermediate representation) STARK over a 64-bit Goldilocks-class prime field, with FRI for polynomial commitment, hash-based lookup arguments, and Fiat–Shamir with BLAKE3. Plonky3/Winterfell-class provers are production-grade open software today; nothing in this section requires new research.

Because the protocol now contains two families of hashing, the discipline is stated once, precisely:

Hash | Where it is used | Why

BLAKE3 | Note commitments, nullifier derivation, txid, headers, C_t, anchors, DA commitments, mixnet MACs, Fiat–Shamir | The external, binding face of the protocol: collision and preimage resistance with fully post-quantum generic bounds; native speed; universal audit familiarity

Poseidon2 over the prover field ($t = 16$, 256-bit capacity, 256-bit digests = 4 field elements) | Internal node hashing of the note commitment tree, nullifier tree, and transit log | Chosen for in-circuit economy: a Poseidon2 permutation costs roughly 800 trace cells inside the STARK versus roughly 2,500–3,000 for a BLAKE3 compression call via lookup tables. Natively it is still microsecond-fast for full nodes

Both are symmetric constructions; both are post-quantum under generic bounds (classical collision resistance 2^{128} , quantum $\approx 2^{85}$ for 256-bit digests; Poseidon2's capacity is sized so that no known algebraic attack beats the generic bound — a standing per-epoch review item, listed in §9.4). Tree digests remain 32 bytes everywhere, so headers, anchors, and witnesses are unchanged in size by the discipline.

The chips, as sub-AIRs with lookup arguments:

Chip | Job | Dominant technique

Merkle chip | Verify 32-deep Poseidon2 membership paths against the anchored root | One permutation per level; ~800 cells per level

Nullifier chip | Derive $nf = \text{BLAKE3}(nk \parallel p)$ | BLAKE3 compression via 16-bit lookup tables

Commitment chip | Verify output $cm = \text{BLAKE3}(v \parallel p \parallel d \parallel r)$ | Same lookup machinery

Range chip | 60-bit value ranges; smallness bounds for seal randomness and noise | Windowed lookup range proofs

Sparse encoder chip | Compute $\delta = W \cdot \Delta S$ over ≤ 11 active features | Column selection by lookup; two-limb fixed-point multiply-accumulate with explicit carry handling (W is 16-bit fixed-point, ΔS is 64-bit; partial products are kept below the field modulus by construction)

Digit-encoder chip | Decompose δ into the canonical 10-bit digits of the seal plaintext | Bit-level decomposition, lookup-windowed

Seal chip | Verify NERV-Seal well-formedness: $u = A \cdot r + e_1$, $v = T \cdot r + e_2 + \text{scale} \cdot \text{digits}$, with small r , e |

In-circuit NTT over the 32-bit seal modulus, emulated in the 64-bit prover field (each coefficient fits in one field element; each modular multiplication is one field multiplication plus a division-check)

Wiring | Fiat–Shamir transcript, binding challenge, blinding rows | —

The spark/injectivity property of W (§7.3) remains out-of-circuit, certified per governance epoch; the circuit checks admissibility and non-zero-ness only.

5.4 The Honest Constraint Budget

For a typical 2-input, 2-output single-shard transaction (per additional leg in a cross-shard transaction, add one Merkle path, one nullifier, one commitment, one feature vector, one delta, one digit encoding, and one seal statement):

Component | Trace cells

Merkle membership (2 paths $\times$ 32 levels, Poseidon2) | $\sim 51,000$

Nullifier derivation (2 $\times$ BLAKE3) | $\sim 7,000$

Output commitments (2 $\times$ BLAKE3) | $\sim 7,000$

Conservation + ranges (4 values) | $\sim 3,000$

Feature construction, admissibility, log rail, digest | $\sim 3,000$

Sparse encoder ($\delta = W \cdot \Delta S$) | $\sim 3,500$

$\delta \neq 0$ | 64

Digit encoding into seal plaintext | $\sim 1,500$

NERV-Seal well-formedness (NTT, smallness, linear relations) | $\sim 90,000$ – $120,000$

Wiring, challenges, blinding | $\sim 3,000$

Total bundle | $\sim 170,000$ – $200,000$

Two honest observations follow. First, the seal dominates: 55–60% of the circuit exists to prove that a ciphertext is a faithful encryption of a value that is itself proven correct. This is the price of replacing Pedersen commitments and decryption oracles with verifiable encryption – and it is paid exactly where the privacy wall stands, which is where a sound design should spend its constraints. Second, wallet proving is a background job, not a magic trick: on a Plonky3-class prover, the full bundle proves in roughly 0.8–2 s on a laptop and 4–12 s on mid-range phones. Proving begins the moment the user signs and overlaps mixnet transit (median ≈ 1.3 – 1.8 s), so the user-perceived submission latency is approximately $\max(\text{proving time, mixnet transit})$, and confirmation is governed by the registry pipeline (§5.5), not by the prover. v2.1's "50 milliseconds on a phone" was true of a delta core that no longer exists in that form; v3.0 states the real number because a protocol that overclaims is a protocol that will be trusted once.

Proof sizes: a transaction STARK at 100-bit post-quantum security is 120–180 KB. This is the honest cost of transparency – v2.1's Halo2 proofs were hundreds of bytes, but they carried elliptic-curve discrete logarithms in every verification. The size burden is discharged by aggregation: amortized to roughly 250–500 bytes per transaction at the bundle tier (§5.5), and to a single proof per second at the consensus tier.

5.5 The Three-Tier Proof Fabric

The protocol never asks a validator to verify a million proofs per second. It asks the network to verify one.

Tier 0 – wallets prove. Each wallet produces its transaction STARK and submits the transaction (shell, encrypted notes, seal ciphertext, proof) through the mixnet or directly to aggregators. The proof is the only place the transaction's secrets ever exist in proven form.

Tier 1 – aggregators fold. An open, staked, permissionless market of aggregators collects transactions, natively verifies each proof (0.2–0.5 ms each on commodity cores), and folds a bundle of 512–4,096 verified proofs into a single bundle STARK. The folding mechanism is amortized recursive FRI verification: the bundle circuit executes the verifier algorithm for every inner proof, batching the inner proofs' FRI queries so that each additional inner proof costs roughly 200,000–400,000 marginal cells. This is

production-proven technology — recursive STARK composition with query amortization is deployed today by commercial STARK provers — and requires no new assumptions beyond those of §5.7. The bundle is submitted to the registry with its transactions and a Merkle commitment to its txid set.

Tier 2 — the registry commits. A rotating registry committee (a beacon-sampled committee, distinct from shard committees) verifies incoming bundles, resolves duplicates canonically (a txid settles into the first interval in which a valid bundle contained it), and commits the interval's transaction set T_τ as a Merkle root, together with one global validity proof G_τ — a second folding level over the interval's bundle proofs. The beacon verifies G_τ (one STARK verification, sub-millisecond) and finalizes the interval. From that instant, every transaction in T_τ is valid protocol-wide, and every shard may settle its legs by Merkle membership witness into T_τ — plain hashing, no per-transaction zero-knowledge verification anywhere in the consensus path (§4.3).

Censorship resistance and omission. Aggregators cannot invalidate; they can only omit. Clients therefore submit to several aggregators simultaneously; the registry must include any valid bundle (deduplication is deterministic by txid); and an omitted transaction is immediately visible to its own submitter, who resubmits. Aggregators post stake, slashed if a bundle they attest fails verification. The registry committee is similarly staked and slashable.

The degraded-mode backstop. If the prover market fails to deliver G_τ within an interval plus a short grace period, the registry committee attests T_τ directly with an ML-DSA quorum certificate, and the beacon finalizes the interval after a 30-second challenge window. Any party may challenge an attested inclusion by presenting the transaction and its proof for deterministic re-verification on the beacon — a fraud proof of one proof. Soundness in degraded mode rests on committee stake plus the deterministic verifiability of every inclusion (an invalid inclusion is always provably invalid); liveness never depends on the prover market's health. This is optimistic validity as a fallback, recursive validity as the steady state — both specified, both implementable.

The prover economy, priced honestly. Folding cost is roughly linear in throughput: at genesis rates (10^4 – 10^5 TPS) the network's aggregation work is 20–500 GPU-equivalents across the open market; at the 1M TPS stretch target, 2,000–5,000 GPUs — a fleet comparable to the validator hardware of leading high-throughput chains, permissionless in entry, and priced into fees: at a proving fee of 10^{-4} NERV per transaction, prover revenue at full load is ~ 100 NERV/s against an infrastructure cost of ~ 1 –5 NERV/s. A folding-scheme upgrade path (field-agnostic AIR folding, Bova-class) that would cut marginal folding cost roughly tenfold is a roadmap research track (§14) — explicitly not a genesis dependency.

5.6 Aggregation Soundness: Two Lemmas

The review's eighth item demanded that aggregation never substitute for validity. The demand is met by ordering: every delta is individually proven before it is ever aggregated, and the aggregation steps are public arithmetic on proven objects. Stated as lemmas (proof obligations in §13):

L1 — Registry validity. If the beacon finalized interval τ carrying a verifying G_τ (or an attestation past its unchallenged window), then every txid in T_τ was accompanied by a transaction STARK that verified against its anchored public inputs. Obligations: FRI soundness, Fiat–Shamir in the QROM, and the Merkle binding of T_τ .

L2 — Aggregate fidelity. Every revealed per-shard aggregate Δ_B is the exact decryption, by verifiable partial decryptions, of the publicly recomputable ciphertext sum $ct_B = \sum ct_{leg}$, where each ct_{leg} is a STARK-proven encryption of a proven delta of a registry-valid transaction. Obligations: NERV-Seal well-formedness (statement 10), public sum recomputation (§4.3), verifiable partial decryption (§6.3), and the digit-decoding arithmetic of §6.3.

Together with the custody theorems T1–T5 of §4.8, these close the review's central question by construction: the chain proves transactions valid against an independently authenticated state, then — and only then — does it add them up.

5.7 Assumptions, Transparency, and the Price Paid

Primitive | Assumption | Quantum-safe | Role

BLAKE3 | Preimage / collision resistance | Yes (generic bounds) | Commitments, tags, headers, Fiat–Shamir, DA

Poseidon2 ($t=16$) | Algebraic hash security above generic bounds | Yes (generic bounds; algebraic analysis standing) | Tree nodes

FRI | Low-degree testing soundness | Yes | Polynomial commitment in all proofs

Fiat–Shamir (BLAKE3) | FS in the quantum random oracle model | Stated assumption | Non-interactivity of all proofs

Lookup arguments | Hash-based (logUp-class) | Yes | Circuit economy

MLWE / MSIS | Lattice problems | Yes | NERV-Seal, DKG, verifiable partials

ML-DSA / ML-KEM | FIPS 204 / 203 | Yes | Signatures, transport, note encryption

What is absent: elliptic-curve discrete logarithms, pairings, Pedersen algebra, BLS aggregation, trusted setups, and ceremonies of any kind. The single non-standard assumption in the stack — Fiat–Shamir in the QROM, the assumption under which every transparent proof system operates — is named here rather than inherited silently, per P9.

The honest trade against v2.1's Halo2:

Property | v2.1 (Halo2 + IPA) | v3.0 (STARK + FRI)

Raw proof size | ~400 B | 120–180 KB

Amortized proof size | — | 250–500 B/tx at bundle tier; one proof per second at consensus tier

Wallet proving (full bundle) | ~300 ms | 0.8–12 s (device-dependent)

Native verification | ~10 ms | 0.2–0.5 ms

Setup | Universal-updatable ceremony (or IPA) | None — transparent

Soundness assumption | Discrete logarithm (Shor-vulnerable) | Hashes + QROM

Quantum adversary who breaks the proof system | Forges transaction validity attestations | Must break BLAKE3 or FRI

What v3.0 bought: every soundness guarantee in the attestation path now survives a quantum adversary, and the chain never depended on a ceremony. What it paid: proving moved from the pocket to a background job. In a payments network whose confirmation path is measured in seconds, a background proof that completes before the user would notice its absence is the correct place to spend that cost — and the alternative was a soundness assumption with an expiry date.

Section 6

6. The Privacy Stack: Mixnet Anonymity and Aggregate-Only Revelation

6.1 Four Walls

NERV's privacy is four independent walls, each defending a different secret, each with a named mechanism and a stated adversary (§6.4):

Wall | Protects | Mechanism | Section

Custody | Amounts, parties, note contents | Note encryption, commitments, nullifiers, ZK proofs | 3, 5

Network | The link between a sender's network identity and their transaction | PQ-Sphinx mixnet | 6.2

Delta channel | Individual transaction deltas — forever | NERV-Seal: linearly homomorphic threshold encryption with in-proof verifiable encryption | 6.3

Revelation | Per-block aggregates from single-transaction leakage | Batch minimums, chunked decryption, lossy projection, committee padding | 6.3, 6.4, 7.4

The design rule across all four (P6): plaintexts that matter are produced by wallets, never by the network. The network's only plaintext outputs are per-shard, per-block aggregates.

6.2 The PQ-Sphinx Mixnet

What it protects, stated precisely: a transaction's public shell is publishable — it contains no addresses, amounts, or linkable metadata. What the mixnet protects is the association between the shell and the IP address that submitted it. Without this wall, a global observer correlates timing and origin across every submission and de-anonymizes users statistically, regardless of the cryptography underneath.

Operation. Wallets select 5 relays from the on-chain relay registry — staked, slashable operators whose ML-DSA-attested ML-KEM-768 public keys are committed on-chain. Selection is VRF-seeded per transaction, diversified by geography and operator. For each hop i , the wallet encapsulates a fresh shared secret (ML-KEM-768), derives a domain-separated key, and wraps the next layer in ChaCha20-Poly1305. Each relay decapsulates, peels one layer, learns only the next hop, and forwards. The final relay submits the transaction to the aggregator mempool; no relay ever sees plaintext — there is none to see.

Packet discipline. All packets are uniform 20 KB Sphinx-format messages — enlarged from v2.1's 8 KB to carry five post-quantum encapsulation headers (≈ 1.2 KB each) plus the transaction payload (shell, notes, seal ciphertext, and headroom for onion layers). Fixed size defeats volume analysis; headers are indistinguishable from random; replay is defeated by per-hop MAC verification with monotonically tagged sequence numbers.

Timing defense. Each relay holds every packet for a delay drawn from an exponential distribution (mean 100 ms, hard-capped at 500 ms, within the $\mu = 100$ ms / $\sigma = 200$ ms jitter envelope). End-to-end ingress latency: median ≈ 1.3 – 1.8 s. This price is paid once, at submission, overlapped with wallet proving (§5.4) — it does not affect block time.

Cover traffic. Relays emit dummy packets — valid Sphinx messages addressed to other relays and discarded at their destination layer — shaped by a local learned traffic model, the same linear-learner pattern as the network forecaster (§10), trained on the relay's own observed traffic. These models are knowledge-layer class: local, non-canonical, never consensus inputs. The honest limitation (P8) is unchanged from v2.1: the network can slash provable misbehavior (dropping, malformed forwarding) but

cannot slash poor modeling; cover-traffic quality depends on relay competence, which stake and reputation price but do not guarantee. Relay software is open-source and audited for this reason.

6.3 NERV-Seal: Complete Specification

The review's thirteenth item demanded that the homomorphic channel be specified end to end — scheme, assumptions, ciphertext structure, aggregation, decryption, threshold construction, key generation, rotation, failure recovery, malicious-participant handling, and proof that aggregation cannot be manipulated. This section is that specification. NERV-Seal is not a new cryptosystem; it is the standard module-LWE construction, parameterized for additive aggregation over a 64-dimensional plaintext, with threshold decryption assembled from the same Fiat–Shamir machinery as ML-DSA. Every component is implementable today from open primitives.

6.3.1 The scheme

Let $R_q = \mathbb{Z}_q[x]/(x^{256} + 1)$ with q a 32-bit NTT-friendly prime ($q \equiv 1 \pmod{512}$). The shard-epoch public key is (A, T) with $A \in R_q^{8 \times 8}$ uniform and $T = A \cdot s + e_0 \in R_q^{2 \times 8}$, for a short secret $s \in R_q^8$ (rank 8, total dimension 2,048). To seal a leg's delta $\delta \in (\mathbb{Z}/2^{64})^{64}$:

1. Digitize: each of the 64 coordinates is decomposed into 7 canonical digits of 10 bits (448 digits; the top digit is a guard), placed in the plaintext polynomial pair $m \in R_q^2$ (512 slots).
2. Sample short $r \in R_q^8$, $e_1 \in R_q^8$, $e_2 \in R_q^2$, with binomial noise and in-circuit-enforced bounds (§6.3.7).
3. Compute the ciphertext $ct = (u, v)$ with $u = A \cdot r + e_1 \in R_q^8$ and $v = T \cdot r + e_2 + \text{scale} \cdot m \in R_q^2$, where scale is a power of two chosen by the noise budget (§6.3.8).

Decryption of an aggregate ciphertext (u_B, v_B) computes $\hat{m} = v_B - s \cdot u_B$ and recovers each digit by rounding $(\hat{m} - \text{center})/\text{scale}$; public carry resolution then reconstructs the coordinate sums modulo 2^{64} exactly.

6.3.2 Aggregation

Addition is component wise polynomial addition modulo q : $ct_B = \sum ct_i$. Because encryption is affine in the plaintext with a fixed scale, the sum decrypts to the sum of digit vectors plus summed noise — and because digit addition is carry-free per slot, the decoder recovers exact digit sums (each bounded well within its headroom for batch sizes up to 512), then resolves carries publicly. Batches larger than 512 are decrypted in chunks; the block aggregate is the public sum of chunks revealed. No elliptic curve, no pairing, no trusted setup appears anywhere in the channel.

6.3.3 Threshold decryption

Each shard's epoch decryption committee (t -of- n , beacon-sampled per epoch; genesis parameters $t = 7$, $n = 10$) holds Shamir shares of s over R_q , generated by the DKG of §6.3.5. For a chunk ciphertext (u_B, v_B) , member j publishes the partial $p_j = \lambda_j \cdot s_j \cdot u_B + \epsilon_j$ with ϵ_j small, together with a Fiat–Shamir lattice Σ -protocol proof (Lyubashevsky-style — the ML-DSA core machinery) attesting that p_j is consistent with its public share commitment and u_B . The proof is 2–5 KB and verifies in tens of microseconds; it is checked by every full node once per chunk. Aggregating the partials yields $s \cdot u_B$ plus bounded noise, and the committee never learns, at any point, any individual transaction's delta — each

member sees only ciphertext sums and its own share. Aggregate-only revelation is thus a property of the construction, not a policy promise.

6.3.4 Why aggregation cannot be manipulated

The review's fourteenth item required a binding chain from transaction to delta to ciphertext to proof. In v3.0 the chain is closed at proof time and audited forever after:

- Each ciphertext is individually bound to its proven delta by STARK statements 9–10 (digit encoding + well-formedness with bounded noise). A malformed or mismatched ciphertext is not provable; an unproven ciphertext cannot settle.
- The batch sum ct_B is publicly recomputable by every full node from the settled legs' ciphertexts — plain modular additions, no trust (§4.3).
- The decryption is verifiable: each partial carries its zero-knowledge proof; the digit decoding is public arithmetic; the reconstructed Δ_B is checkably the exact decrypt of the checkable sum of checkable ciphertexts.

There is no step at which a producer, committee member, or aggregator can substitute, inflate, or poison an aggregate without producing a publicly verifiable fraud condition. This is what replaces v2.1's Pedersen audit wall — and unlike Pedersen, every assumption under it is lattice- or hash-based.

6.3.5 Distributed key generation

Per shard, per epoch, the committee runs a dealerless DKG over R_q : each member publishes Shamir coefficient commitments in the Ajtai form $c = A^t \cdot [x; \rho]$ (MSIS-based, linear, post-quantum — the algebra of §3.5's standalone-commitment row) with Fiat–Shamir consistency proofs; the aggregate public key T is the sum of share commitments; the transcript is committed on the beacon under the shard's `params_root`. Anyone can verify the DKG from public data; no dealer, no trusted hardware, no elliptic curve.

6.3.6 Rotation and failure recovery

Epochs last 24 hours. At rotation, the outgoing committee proactively re-shares the joint secret to the incoming committee (verifiable resharing, with encrypted backup shares under ML-KEM to the incoming members) before disbanding — this handoff preserves the ability to complete pending old-key decryptions across the boundary. Sub-minimum pending batches at epoch end are force-padded with committee-generated dummy ciphertexts (well-formed encryptions of zero) to the batch minimum and revealed, so no value lingers unopened. Emergency re-key under governance follows the same forced-reveal-then-reshare path.

Failure honesty (P9, applied inward): if a committee loses quorum irrecoverably mid-epoch, that epoch's pending aggregates are lost. The embedding records a governance-logged gap; the forecaster's replay skips the interval; custody is untouched — no coin, balance, or proof depends on a reveal. This bounded failure mode is the deliberate price of the channel's advisory tier, and it is why the channel sits outside C_t .

6.3.7 Malicious participants and noise bounds

Invalid partial decryptions are rejected by their own zero-knowledge proofs, and the faulty proof is slashable evidence. DKG lies are caught by consistency proofs. Ciphertext griefing is impossible by statement 10. Collusion of $\geq t$ committee members can, in principle, decrypt individual ciphertexts — this is inherent to threshold public-key encryption, was stated in the threat model of §2.5, and is bounded by per-epoch sampling, stake, ex-post forensics, and the fact that individual deltas are rank-64 projections of already-private data (the advisory tier's 96-bit parameter posture, bounded in §9.4). Finally, the noise vectors are not merely sampled small — their smallness is proven in-circuit (range-checked against 3σ bounds with wallet-side rejection sampling), which converts the decoding-failure analysis from a probabilistic hope into a proven per-ciphertext property, with chunk sizes and scale chosen so that worst-case accumulated noise remains well inside the decoding margin.

6.3.8 Parameters, sizes, and honest accounting

Parameter | Genesis value

Ring | $R_q = \mathbb{Z}_q[x]/(x^{256}+1)$, q a 32-bit NTT prime

Module rank / dimension | 8 / 2,048

Plaintext rings | 2 (512 digit slots; 448 used)

Digit size | 10 bits; 7 digits per coordinate (guard digit included)

Noise | Binomial $\eta = 2$, in-circuit 3σ bounds

Chunk size | ≥ 128 (privacy floor) and ≤ 512 (noise ceiling)

Security target | ≥ 128 -bit classical, ≥ 96 -bit quantum per the standard lattice estimator — the advisory tier of §2.6; final freeze is an audit deliverable (§14)

Ciphertext size | ≈ 8 –10 KB at genesis wire format (u dominates); the 4–6 KB figure of the compression roadmap (§14) is achieved by coefficient rounding of v (always-on) and u (roadmap)

Public key | ≈ 8 KB per shard per epoch, committed once under `params_root`

§3.6 quoted 4–5 KB for the sealed delta; this section's genesis number is larger. The discrepancy is stated rather than hidden (P9): the earlier figure is the compressed target, this one is the genesis wire format. The seal is the single largest per-transaction object and the network's dominant transient bandwidth cost (~ 10 KB/tx) — which is precisely why its data placement is engineered: individual ciphertexts live in the erasure-coded data-availability layer (the layer designed for bulk), headers commit only their hashes and the batch sum, and a producer-side aggregation proof that would permit DA pruning of individual ciphertexts entirely is a specified roadmap optimization (§14). Permanent state grows by ~ 32 bytes per transaction from the seal, not ~ 10 KB.

6.3.9 Alternatives considered — the first-principles ledger on Pedersen's replacement

The review asked for serious alternatives to Pedersen commitments; §3.5 set the assignment policy and this section closes the argument. Pedersen's binding rests on discrete logarithms and falls to Shor — an unacceptable foundation for the object that audits everything else. Hash commitments are perfectly post-quantum but not homomorphic, so they cannot carry the aggregate channel. Standalone Ajtai commitments are homomorphic and post-quantum, and NERV-Seal's $T \cdot r$ relation is exactly that algebra — but deploying a detached Ajtai commitment alongside the ciphertext would double the per-transaction object count to buy hiding the channel does not need (the wall's height is already set by LWE, per the composite-wall principle of §3.5). The sound alternative to Pedersen is therefore not another commitment scheme bolted beside the ciphertext — it is the ciphertext itself, made binding by verifiable

encryption inside an all-hash, all-lattice proof system. One object, three jobs: transport, binding, audit. The protocol deletes a primitive and gains a quantum adversary it can shrug at.

6.4 Adversary Models and Privacy Properties

NERV claims privacy per property, per mechanism, per adversary — never as a percentage (review items 10, 11, 29).

Property | Definition | Mechanism | Named adversary and bound

Transaction confidentiality | Amounts, parties, and note contents are computationally hidden from validators and observers | ML-KEM-768 + ChaCha20-Poly1305 note encryption; STARK zero-knowledge (trace blinding; formal audit item §13 E) | Any classical or quantum adversary bounded by ML-KEM and hash security; IND-CCA in the standard model for the symmetric part

Sender–receiver unlinkability (data layer) | No efficient adversary links a transaction's inputs, outputs, or counterparties from chain data | Pseudorandom nullifiers and commitments; diversified addresses; sealed deltas | Any observer of all public data, forever — bounded by BLAKE3 preimage resistance and LWE

Amount privacy | Values never appear in the clear | Commitments + in-circuit ranges | As above

Network anonymity | Sender–transaction linkage is hard despite traffic observation | 5-hop PQ-Sphinx, exponential jitter, cover traffic | An adversary observing fraction f of network links and controlling fraction $g < \frac{1}{3}$ of relays: expected anonymity set $\geq (1-f)^2 \cdot (1-g)^5 \cdot N$, where N is the submission count in the de-anonymization window. At $f = g = 0.1$, this is $\approx 0.4 \cdot N$; the guarantee therefore scales honestly with adoption — at 10^5 submissions per window, anonymity sets on the order of 4×10^4 ; the 10^5 – 10^6 targets are reached at sustained load. Timing residual is bounded by the cascade's jitter entropy; these are simulation-validated design targets (§14), not unconditional guarantees

Aggregate privacy | Individual deltas are not inferred from revealed aggregates | Batch minimum $B_{\min} = 128$ (raised from v2.1's 64), chunked decryption ≥ 128 , committee padding, roll-forward, and the lossy rank-64 projection | An adversary seeing all reveals: per-transaction inference is bounded by the projection's conditioning mixed across ≥ 128 contributions; the aggregate time series itself is public by design, and a formal leakage analysis of $(W, B_{\min})$ is a standing governance review item

Metadata resistance | Side channels minimized | Fixed 20 KB packets, uniform jitter, fee values public by necessity (an honest residual: fees are a coarse priority signal; wallet-side fee bucketing mitigates and is optional)

What NERV does not claim: "100% privacy" appears nowhere. An adversary exceeding the network-anonymity model (observing most links, or confirming traffic against a low-traffic target) degrades the network wall without touching the data-layer walls; a future quantum adversary exceeding 96-bit effort against MLWE can, at most stated cost, attack the advisory delta channel — and still learns nothing the custody wall protects. Every such residual is priced, named, and bounded, because a protocol that overclaims is a protocol that will be trusted once.

The next section descends from the walls to the machinery they shelter: the neural overlay, demoted to a provable accelerator — the codec W , the embedding as a derived index with its collision theorem stated rather than hidden, injectivity as index-completeness rather than security — followed by the first-class sharding architecture with its cross-shard transit rules and honest end-to-end throughput budgets, the

complete post-quantum audit of every primitive in the stack, and the knowledge layer that learns in public without ever touching authority.

Section 7

7. The Neural Overlay: A Provable Accelerator, Demoted

7.1 What Changed in Status

In v2.1, the neural weight codec W and the state embedding e_t lived inside the state root and stood at the semantic center of the architecture. In v3.0 they survive with every one of their computational virtues — exactness, $O(1)$ updates, tiny circuits, a learning substrate — and with none of their former authority. The review's fifth item drew the boundary, and this section enforces it:

The cryptographic commitment C_t is the source of truth. The embedding e_t is a public, derived index — a mirror of the economy computed from revealed aggregates, committed as derived data under D_t , consulted by nothing canonical.

The clearest way to state the demotion is a test the protocol must pass by construction, not by policy: the delete test. Delete every embedding, every forecast, every residual, and every model parameter from every header, every archive, and every node. Recompute C_0 forward. Every note, every balance, every proof, every inclusion witness, and every theorem of Section 4 remains valid and verifiable. The overlay is removable silicon — the ledger's skull does not need its sensory cortex.

7.2 The Codec W and Per-Leg Feature Vectors

The encoder remains the smallest learner in existence: a single-layer linear map, no bias, no non-linearity:

$$\delta_{\text{leg}} = W \cdot \Delta S_{\text{leg}} \in (\mathbb{Z}/2^{64})^{64}$$

with $W \in (\mathbb{Z}/2^{16})^{64 \times 256}$ — 64 output dimensions $\times$ 256 input features, 16-bit signed fixed-point weights (1.15 format), quantized and hash-committed at each governance epoch. The affine-term issue the review carried forward from v2.0 (item 9) remains resolved by construction: the canonical map has zero bias, so additivity is exact by algebra, and the forecasting layer's bias parameter lives entirely in the advisory tier where it can be retrained freely.

Feature vectors are per leg (§3.6). Each leg's ΔS_{leg} is constructed client-side from that leg's own data:

Features | Indices | Content

Account slots | 0–223 (224 slots) | For each input homed in this shard: $-v$ at slot(spending address). For each output homed here: $+v$ at slot(recipient's delivery address). $\text{slot}(\text{addr}) = \text{BLAKE3}(\text{"nerv.slot"} \parallel \text{addr}) \bmod 224$ — deterministic, client-computable, provable in-circuit via lookup

Volume rail | 224 | Total output value of this leg (nano-NERV)

Fee rail | 225 | This leg's declared fee

Count rail | 226 | 1 (marks a leg event)

Log-magnitude rail | 227 | $\lfloor \log_2(1 + v) \rfloor$ per value via circuit lookup (features may be non-linear transforms of transaction data; only the map W must be linear)

Type/time rails | 228–255 | Transaction-type flags, coarse time-of-epoch buckets (public, not private timing)

Sparsity: an ordinary leg activates at most 6 account slots plus a handful of rails, so the dot product reduces to selecting and scaling at most 11 of W 's 256 columns — roughly 3,500 trace cells in the STARK (§5.4). Slot collisions are harmless: two addresses hashing to the same slot blur the public index slightly and nothing else, because ownership and solvency live entirely in Section 3.

7.3 The Collision Theorem, Stated Instead of Hidden

The review's third item is not a bug report; it is a theorem, and v3.0 states it as one:

Theorem (informal, E1). The embedding map is not injective on ledger states. The embedding carries $64 \times 64 = 4,096$ bits, hence at most 2^{4096} distinct values; the set of reachable ledger states is vastly larger and grows without bound. Therefore there exist materially different states $S_1 \neq S_2$ with identical embeddings, and this cannot be repaired by any choice of W , any training, or any parameter regime.

Predecessor designs treated this as an embarrassment to be engineered around. v3.0 treats it as the precise reason the architecture was inverted: a fixed-dimensional linear projection is information-theoretically disqualified from binding an unbounded ledger, so it is never asked to. The consequence is architectural, and it has already been paid in Section 4 — the commitment C_t is hash-based and unbounded; the embedding is downstream; nothing authoritative consults it.

7.4 What $\delta \neq 0$ and Injectivity Actually Provide

The review's fourth item correctly observed that v2.1's $\delta \neq 0$ check is a narrow protection, not collision resistance. v3.0 keeps both v2.1 defenses with honestly-scoped names:

- Non-zero-ness (proof statement 8): a sanity check that every provable transaction moves the public index. It is 64 cheap constraints and is described nowhere in this document as a security property.
- Injectivity on the admissible support (index completeness): the circuit enforces that every ΔS_{leg} has at most 6 active account slots; the account-slot block of W is required to be injective on that support, certified per governance epoch by the spark condition (any 13 of the 224 slot columns linearly independent; $k_{\text{max}} = 6$ active slots means differences involve at most 12 columns; a generic random 64×224 matrix has spark 65, so verifiable random generation clears the bound with overwhelming margin). What this buys is completeness of the index: every genuine, provable transfer moves the fingerprint, so light analytics and the forecaster never miss real settled activity. What it does not buy — stated plainly — is collision resistance over states, unforgeability, or any custody property. Those live in Section 3 and Section 4, under hashes.

7.5 The Embedding as a Derived Public Index

Each shard maintains its embedding exactly as before, minus the authority:

$$e_{t+1} = e_t + \Delta_B \pmod{2^{64}}$$

where Δ_B is the revealed, verifiably-decrypted aggregate of the block's settled legs (summed across the block's decryption chunks — §6.3). Wraparound arithmetic keeps the accumulator a closed group: no overflow, no saturation, no drift, exact replay. The embedding, its full history, and the forecaster's state are committed under D_t (§4.2), a derived root whose recomputation anyone can audit and whose disagreement with any implementation is a public determinism bug in the advisory layer — slashable, and irrelevant to custody.

7.6 Weight Versioning and the Amnesia Rule

W is frozen between epochs (P_3) and changes only through the slow governance ceremony: a fresh randomness beacon produces candidate W' ; the epoch certificate verifies quantization format, weight norms, the spark bound, and column–rail independence; stake-weighted governance adopts W' with an activation height; params_root updates to commit $\text{Hash}(W' \parallel \text{version})$.

The Amnesia Rule — the state is never re-encoded — is retained unchanged, because it was correct in v2.1: at the epoch boundary, e_t carries forward; new deltas are computed under W' and added on; each delta proof pins its own W -version; old receipts and witnesses remain valid forever. The embedding becomes a layered record, each epoch's increments written in that epoch's coordinates — sound because the accumulator only ever needs addition to be exact. The forecaster re-initializes at every boundary (optimizer state resets; warm-start only on post-boundary aggregates), since a change of W changes the coordinate system of the learning target.

7.7 What the Embedding Is — and Is Not

Is: a 512-byte derived public accumulator; a lossy, linear, injective-on-admissible-support index of settled economic activity; $O(1)$ -updatable; exactly provable inside a transparent STARK; the substrate on which the knowledge layer learns (Section 10); removable without consequence.

Is not: the ledger of record (notes, nullifiers, and C_t are); a source of authority for any balance or validity decision; a commitment to anything (E_1 forbids it); a privacy mechanism of the same tier as the note layer (its privacy is aggregate and computational — §6.4); a learned artifact.

The crown property, restated as the section's close: corrupting W — totally, catastrophically, adversarially — degrades the quality of a public index and a forecast. It cannot mint a note, spend a note, double-spend a note, forge a quorum certificate, or unbalance a single conservation equation. Money lives in cryptography; the embedding lives in linear algebra; and in v3.0, only the second of those is allowed to have an opinion — about anything.

Section 8

8. Sharding: First-Class, Deterministic, Cross-Shard by Construction

8.1 Principles

The review's seventeenth item demanded that sharding be defined independently of the embedding — a real partition function, with specified object ownership, migration, cross-shard semantics, atomicity, proof

generation, receipts, replay prevention, finality, validator assignment, and data availability. This section is that subsystem. Its principles:

- Deterministic homing. Every address belongs to exactly one shard by a public rule; every note belongs to the shard of its delivery address; every leg settles in exactly one shard.
- No coerced migration. State never moves under protocol force. Users move their own value with ordinary transactions; the topology changes only where new addresses are born.
- Atomicity by construction, not coordination. Cross-shard semantics are the transit rules of §4.5 — deterministic, deadlock-free, permissionlessly completable.
- Advisory forecasts, deterministic decisions. Machine-learned load estimates may propose; only deterministic threshold rules evaluated on finalized public data dispose (review items 18–19).
- Horizontal everything. Bandwidth, execution, proving, decryption, and DA are all per-shard; global services (beacon, registry) are $O(1)$ in shard count for every validator.

8.2 The Shard Trie and Address Homing

Section 3.2 gave the simplified homing rule $\text{shard}(\text{addr}) = \text{BLAKE3}(\text{"nerv.shard"} \parallel \text{addr}) \bmod N_s$. The full rule — the one the protocol actually runs — is prefix-based, and the difference is load-bearing for dynamic topology:

An address's shard key is the bit string $\kappa(\text{addr}) =$ the first 256 bits of $\text{BLAKE3}(\text{"nerv.shard"} \parallel \text{addr})$. The active shard set is a prefix partition: a set of binary shard IDs, no one a prefix of another, whose union covers all bit strings. An address is homed to the unique active shard whose ID is a prefix of $\kappa(\text{addr})$.

Genesis activates the 64 shards with 6-bit IDs (000000 through 111111). A split replaces shard p with $p \parallel 0$ and $p \parallel 1$; a merge replaces siblings $p \parallel 0$ and $p \parallel 1$ with p . Because homing is by longest-prefix match over an explicit ID set, splits and merges never re-home existing notes — they only change where new addresses are born. The shard tag travels with the address and appears in leg placement, so — as §8.5 details — no hidden information is ever needed to re-partition anything.

8.3 Committees, Sampling, and the Beacon

Validators stake globally (transparent staking records on the beacon — accountability is a feature, not a leak, for validators) and are assigned by VRF-sampled sortition per epoch (24 hours):

Role | Size (genesis) | Tolerance | Notes

Shard committee | 21 | 7 Byzantine | Produces shard blocks; ML-DSA quorum certificates; also serves as the epoch decryption committee (§6.3) with independent threshold $t = 7$

Beacon committee | 31 | 10 Byzantine | Finalizes shard headers, the registry, and interval attestations; rotates in staggered thirds

Registry committee | 21 | 7 Byzantine | Verifies bundles, commits T_τ and G_τ (§5.5); distinct from shard committees

Sampling is unpredictable before the epoch, small enough to be cheap, large enough that adaptive corruption within an epoch is impractical at the stated stakes. A shard's committee validates its shard natively (all public hashing — §4.3) plus one registry-proof verification per interval. Cross-shard facts arrive as beacon-finalized data plus Merkle witnesses: no shard ever waits on another shard's committee.

8.4 Cross-Shard Recapitulation

The full cross-shard protocol was specified where it belongs, in the ledger: spend legs settle unconditionally; issue legs settle once their siblings' settlement is beacon-finalized (§4.5); verification is a witness check against a finalized root; completion is permissionless — the recipient's wallet, a relayer market (paid from the transaction's fee shares, competing on inclusion latency), or any altruist can submit an issue leg; atomicity, deadlock-freedom, liveness, and censorship honesty are lemmas T5–T8 of §13. What Section 8 adds is the economic layer: relayers are an open market, issue legs are public data in the registry, and the protocol charges nothing for cross-shard settlement beyond ordinary leg fees — cross-shard is not a premium feature bolted onto a single-shard chain; it is the same machinery with two legs instead of one.

8.5 Dynamic Topology: Deterministic Rules, Advisory Forecasts

Splits and merges are governed by deterministic hysteresis on finalized metrics, with the forecaster strictly advisory — the exact pattern the review's eighteenth item prescribed:

Forecast → deterministic rule → consensus proposal → finalized protocol decision

Split rule. If a shard's trailing 7-day average of settled legs per second exceeds the split threshold S_{hi} (genesis: 4,000/s — half of the shard's engineered sustained ceiling), the split is proposed automatically at the next epoch boundary. Anyone may trigger the proposal by presenting the finalized metric; the rule, not judgment, decides.

Merge rule. If sibling shards both sit below S_{lo} (genesis: 400/s) for 30 consecutive days, a merge proposal activates the parent for new addresses.

Adoption. Routine proposals execute automatically at epoch boundaries with a 7-day notice window (so wallets begin diversifying new addresses into the children immediately). Emergency conditions (sustained overload beyond $2\times$ ceiling) allow governance to accelerate. All thresholds are governance parameters; all decisions are computed from finalized public data by every node identically.

Migration is permissionless and economic. Existing notes never move. A split's effect is that new addresses — and therefore new economic activity — distribute across the children. Users migrate proactively by ordinary self-spends into fresh addresses in under-loaded children (a self-spend is the standard re-homing instrument; no special transaction type exists or is needed). Legacy shards remain fully operational forever — their trees, nullifier sets, transit logs, and embeddings continue under their existing IDs — but stop receiving new addresses, so their load decays as the economy flows to the leaves. Governance may formally retire a legacy shard only after a sunset audit demonstrating negligible residual activity, and even then the retirement is a bookkeeping event (the shard's structures become archival), not a coercion: unspent legacy notes remain spendable, forever, through the standard anchor mechanism.

No hidden-bit problem exists in this design, and it is worth saying why, because it is the trap that prefix-based privacy shards usually fall into: re-partitioning a shard's note tree by address bits would require the addresses, which are private. NERV never re-partitions anything. Trees are append-only and permanent; topology governs only the homing of new addresses; the shard tag of an address is public by construction (it is leg placement) and reveals only what leg placement already reveals — the prefix of a hash of a single-use delivery address, which links to nothing.

8.6 Load Balancing and Hot Shards

NERV does not claim perfect load balance and does not need it. Adversarial hot-spotting (flooding one shard) is absorbed by per-shard fee markets: a hot shard's fees rise, wallets route around it, and the attack pays market rate for the resources it consumes. Legitimate imbalance is absorbed by recipient-side diversification: wallets maintain delivery addresses across a rotating set of shards (8 by default, chosen by the wallet's own randomness) and publish the set; senders choose among a recipient's addresses by default-random selection (a mild privacy note: the sender's shard choice among a recipient's published addresses is itself randomized to avoid a selection side-channel). Because addresses are free, diversification is costless. The residual — a genuinely single-shard-bound economy, such as one dominant marketplace whose users never diversify — is priced by that shard's fee market and bounded by that shard's ceiling, exactly as a single-shard chain would price it, and is visible in the public per-shard load series that the embedding's aggregates make available.

8.7 Global Services: Beacon, Registry, Data Availability

The beacon chain (one-second intervals) hosts the three global objects: the finalized header root G_t (a Merkle root over all shard headers — the single anchor a light client verifies), the global validity registry (T_t, G_t — §5.5), and the interval attestations (§11). Its own state is kilobytes; its bandwidth is signatures and roots; its committee is small and staggered. It executes nothing user-facing and can never become a throughput bottleneck by construction.

Data availability is per-shard, hash-committed, and sampled: each shard's block data (legs, ciphertexts, encrypted notes, certificates) is erasure-coded (2D Reed–Solomon) and gossiped as sampled blobs; the shard header commits KZG-free, purely hash-based row/column roots to the DA layer, and light validators sample. Availability is verifiable by any party with a random-sample oracle: unless the adversary withholds $\geq \frac{1}{3}$ of samples, missing data is provably reconstructible — and any reconstructible gap in a finalized header is a slashable, fraud-provable event.

8.8 The Honest Throughput Budget

The review's fifteenth and sixteenth items asked for end-to-end accounting rather than extrapolation from matrix arithmetic. Per transaction, at genesis wire formats:

Component | Transient (DA layer) | Permanent (state/archival)

Shell (nullifiers, commitments, fees, refs) | ~0.7 KB | ~40 B (tree leaves, amortized)

Encrypted notes | ~1.3 KB | — (prunable after recipient scan)

Sealed delta (NERV-Seal, genesis format) | ~10 KB | ~32 B (hash commitment)

Amortized proof (bundle tier) | 0.25–0.5 KB | –

Witness-layer amortization | – | ~0.1 KB

Total | ~12–13 KB | ~0.2 KB

Per shard-second at 10,000 legs/s (the genesis engineered ceiling per shard): ~125 MB/s of transient DA traffic and ~2 MB/s of permanent growth – modest, single-machine-class bandwidth. The numbers that matter for the horizontal path:

Regime | Shards | Aggregate TPS | DA bandwidth (network) | Per-validator bandwidth | Proving fleet | Permanent growth

Genesis | 64 | 250k–640k (sustained–ceiling) | ~0.8–8 GB/s | ~12 MB/s (own shard) + sampling | 100–500 GPUs | ~0.5 GB/s

Design target | 256 | 1M–2.5M | ~12–30 GB/s | unchanged per shard | 2,000–5,000 GPUs | ~2–5 GB/s

Stretch | 1,024 | 4M+ | ~50–120 GB/s | unchanged per shard | 8,000–20,000 GPUs | ~8–20 GB/s

Three honest observations, per P9. First, the per-validator number is the invariant: a shard validator's load is bounded by its shard, whatever the network does in aggregate – this is the entire content of "horizontal scalability," and it is true by construction here. Second, aggregate-class quantities (DA bandwidth, permanent growth, proving fleets) scale linearly with throughput and are carried by open markets (DA relays, archival nodes, the prover market), not by consensus participants. Third, sustained million-TPS operation implies multiple petabytes of archival data per year; the protocol's answer is the separation already in the table – hot state stays small (roots, recent subtrees, nullifier trees are the growth term, and nullifier-set compaction is a governance research item, §14) while archival data is prunable and reconstructible from erasure-coded history held by an incentivized archival market. These are engineering targets with their costs stated, not benchmarks, and the milestone plan of Section 14 sequences them explicitly.

8.9 What One Million TPS Actually Requires

For the record, because the review asked for exactly this enumeration: parallel transaction ingress (the mixnet and aggregator mempools – horizontally scalable relay/aggregator fleets); parallel execution (per-shard, natively parallel hashing pipelines); parallel ZK proving (the open prover market, §§5.5, 14); recursive aggregation (two-tier, already specified); efficient data availability (per-shard erasure coding, already specified); efficient consensus communication (ML-DSA quorum certificates compressed per interval – §11); efficient cross-shard flow (the transit rules – no coordination, no locks); and careful bandwidth engineering (the budget above). Every item on the list exists in this document as a specified subsystem. None of them is "make the matrix multiply faster" – the matrix multiply was never the bottleneck, and v3.0 never pretended otherwise.

Section 9

9. The Post-Quantum Audit: Security by Construction

9.1 Two Distinctions the Review Drew, and How They Resolve

The review's twenty-first and twenty-second items drew two lines that v2.1 crossed. First: PQ user security is not PQ blockchain security — a chain whose users sign with Dilithium but whose validators attest with BLS is a chain whose consensus a quantum adversary can forge while every individual balance remains "quantum-safe" in the brochure. Second: a migration plan is not PQ security at genesis. Both lines are resolved the same way in v3.0: there is nothing to migrate, because nothing quantum-vulnerable was ever installed. BLS is gone (ML-DSA quorum certificates, hash-compressed per interval). Pedersen is gone (§3.5). Halo2 and its discrete-log inner product arguments are gone (all-FRI STARKs). What remains is the inventory below — every security-critical primitive in the protocol, its assumption, and its quantum status.

9.2 The Complete Primitive Inventory

Primitive | Role | Assumption | PQ status | Parameters

BLAKE3 | Note commitments, nullifier derivation, txid, headers, C_t , G_t , DA commitments, mixnet MACs, Fiat-Shamir | Preimage / collision resistance | Yes — classical collision 2^{128} ; quantum $\geq 2^{85}$ under memory-discounted BHT collision search on 256-bit digests | 256-bit digests

Poseidon2 ($t = 16$) | In-circuit tree hashing (NCT, nullifier, transit) | Generic hash bounds; algebraic analysis standing (§9.4) | Yes — capacity sized so known algebraic attacks do not beat generic bounds; per-epoch review item | 256-bit capacity

ML-DSA (Dilithium3, FIPS 204) | Transaction authorization, quorum certificates, relay/committee identity, attestations | Module-LWE / Module-SIS | Yes | NIST level 3

ML-KEM-768 (FIPS 203) | Note encryption, mixnet onion, DKG backup shares | Module-LWE | Yes | NIST level 3

ChaCha20-Poly1305 | Symmetric encryption everywhere | Standard symmetric assumptions | Yes | 256-bit keys

NERV-Seal | Sealed delta channel: transport, binding, audit | Module-LWE / Module-SIS | Yes — advisory tier: ≥ 128 -bit classical / ≥ 96 -bit quantum per the standard lattice estimator; final freeze is an audit deliverable (§14) | §6.3.8

FRI | Polynomial commitment in every proof | Low-degree testing / hash binding | Yes | 100-bit PQ soundness per proof

Fiat-Shamir (BLAKE3) | Non-interactivity of all proofs and Σ -protocols | FS in the quantum random oracle model | Named assumption — the protocol's single non-standard assumption, stated rather than inherited (P9) | QROM

Shamir over R_q + Ajtai commitments | Threshold secret sharing and DKG | Module-SIS | Yes | Aligned with NERV-Seal

VRF (lattice-based, ML-DSA-derived) | Sortition, relay selection | Module-SIS | Yes | —

9.3 The Residual Assumption, Honestly Priced

Fiat–Shamir in the QROM is the assumption under which every transparent proof system in deployment operates today; it is widely regarded as conservative, but it is not a standard-model theorem, and v3.0 names it in every place it is load-bearing (proof soundness L1, verifiable decryption, DKG consistency, the VRF). The audit posture is therefore exactly stated: an adversary who breaks NERV must either invert BLAKE3 or Poseidon2 at generic-cost quantum bounds, solve module-LWE or module-SIS at parameter strength, or refute Fiat–Shamir-in-the-QROM as instantiated here. There is no fourth door, because there is no elliptic curve, no pairing, and no trusted setup anywhere in the stack.

9.4 Cryptanalytic Governance: Standing Reviews and Fallbacks

Because security is a process, not a snapshot:

- Standing reviews (every governance epoch): Poseidon2 algebraic analysis versus generic bounds; the lattice estimator run against NERV-Seal's parameters at both tiers; spark certification of W; the (W, B_min) leakage analysis of §6.4.
- Hash fallback: if 256-bit hash outputs ever approach practical quantum collision, the composite commitment widens by governance to 384-bit digests at a coordinated upgrade — a mechanical change, since every commitment call site is domain-separated and version-tagged.
- Signature fallback: SLH-DSA (FIPS 205) is specified as the emergency validator-signature alternative; quorum certificate sizes grow (hash-based signatures are tens of kilobytes), the interval-attestation compression of §11 absorbs the cost, and the protocol keeps functioning.
- Lattice fallback and the advisory-tier payoff: if module-LWE were ever threatened at NERV-Seal's parameters, the seal channel is the only thing at risk — and it is advisory. The specified emergency action is a governance kill-switch that suspends sealed-delta settlement and reveal: the ledger continues (C_t never consulted the channel), the embedding records governance-logged gaps, forecasting pauses, and every coin is untouched. This bounded failure mode is not an accident of the v3.0 redesign — it is the redesign's payoff, and the reason the channel was moved outside the commitment in the first place.

Section 10

10. The Knowledge Layer: Learning in Public, Touching Nothing

10.1 Purpose and Firewall

The knowledge layer is the network's expectations engine: a per-shard forecaster F that predicts each block's aggregate activity before it is decrypted, is scored after, and updates deterministically. Its purpose is operational intelligence — load anticipation for the shard rules of §8.5, cover-traffic shaping, anomaly visibility — and demonstrable, auditable out-of-sample improvement. Its firewall is absolute, and restated as the layer's first sentence because the review's nineteenth and twentieth items asked exactly this: no consensus outcome, balance, proof, witness, or validity decision depends on any model, any forecast, any residual, or any optimizer state. The forecaster is a deterministic replay of public data, committed under D_t — derived state, never input state. Two validators whose model states diverge have a public, slashable determinism bug in the advisory layer, and custody does not notice.

10.2 The Forecaster

Architecture. Per shard: a linear autoregressive model over the shard's revealed aggregate history — F 's input is a window of the last 1,024 (Δ_B , time-bucket, fee-sum) triples, its output the next block's predicted aggregate $\hat{\Delta}_B \in (\mathbb{Z}/2^{64})^{64}$, plus a per-dimension bias vector c and a scale estimate. Small is a design virtue, not a compromise: the model must be replayable by every node, every block, with bit-exact determinism, and the learning target is a 64-dimensional public time series — the problem does not want a transformer, and a transformer could not survive the determinism requirements anyway.

Deterministic training (review item 20, satisfied point by point). Every operation is integer fixed-point arithmetic with: specified bit widths and rounding (round-half-to-even, applied at named points); canonical gradient ordering (sorted by parameter index); fixed epsilon representation; saturating clipping at published bounds; canonical serialization (little-endian, fixed widths, no floating point anywhere, ever). Adam-in-fixed-point is specified as a reference implementation with golden test vectors in the protocol's conformance suite; a node whose replay diverges from the committed D_t chain is faulted by that public fact alone.

10.3 The Per-Block Loop

1. Commit. Before the block's decryption ceremony, the header commits $H(\hat{\Delta}_B)$, where $\hat{\Delta}_B$ is the deterministic output of F on the previous derived state. Since F is deterministic, the commit is a binding formality — its purpose is to pin the header to the prediction irrevocably before reality exists in plaintext anywhere (the exam is sealed by the channel, not by the hash; the hash makes the sealing auditable).
2. Reveal. The committee opens the aggregate Δ_B (§6.3) — exogenous reality, unknown and cryptographically sealed at commit time. Nothing is ever trained on its own output (P4): the learning target is reality, and the channel guarantees the target did not exist when the prediction was made. This is the property whose absence killed attempt 2, and it survives from v2.1 unchanged because it was correct.
3. Score. The committed prediction is recomputed and checked against $H(\hat{\Delta}_B)$ (mismatch = determinism fault). The residual $r = \Delta_B - \hat{\Delta}_B$ — the network's public, non-repudiable surprise — is recorded in derived state. Huber loss with published per-dimension scales computes the update's blame; tail-residual events raise advisory anomaly flags (consumed by operations and review, never by consensus, never by slashing).
4. Update. F 's parameters update by clipped fixed-point Adam; the new derived state is committed under D_{t+1} . Every step is public, replayable, and boring — which is the point.

10.4 The Challenger Market

An open market pays for beating the incumbent, out of sample, on sealed targets:

- Any account may register a challenger forecaster (a parameterization within the published architecture class) and, each block, commit $\text{hash}(\hat{\Delta}_B)$ before reveal — the same sealed-exam discipline as the incumbent.
- A promotion gate evaluates challengers over rolling 2,016-block windows (approximately two weeks): a challenger that beats the incumbent's Huber score by a governance-set margin, with at least 90% commit

coverage, becomes the incumbent at the next epoch boundary (the incumbent reverts to the reference parameterization at every W-epoch anyway — §7.6).

- Rewards flow only from demonstrated out-of-sample skill on committed-then-revealed targets (P7) — never for in-sample loss reduction, which is indistinguishable from curve-fitting and was the subsidy failure of attempt 2.
- Collusion resistance is inherited from the channel: a challenger cannot see the target before committing, because no one can.

The reward pool for this market is deliberately small — 3% of supply, era-locked (Section 12) — because the layer is deliberately advisory. Self-evolution is a bonus the network earns in public; it was never a pillar, and the v3.0 tokenomics finally say so with numbers.

10.5 Consumers

Shard topology. F's load forecasts feed the deterministic hysteresis rules of §8.5 as inputs to proposals only — forecasts propose, thresholds on finalized data dispose, governance notices, the trie updates. If every forecast were replaced with a constant, the protocol would still function and still scale; it would simply react to load after the fact instead of before.

Cover traffic. Relay-local linear learners (the same pattern, knowledge-layer class) shape dummy traffic to observed distributions (§6.2) — local, non-canonical, priced by relay reputation.

Anomaly visibility. Tail residuals publish advisory flags — early, public signals of spam cascades, MEV spikes, or unusual concentration — consumed by operators, researchers, and governance review. The ledger itself never moves; only expectations do.

The remaining sections complete the specification: the receipt reform — one recursive batch proof, one compact batch-level attestation, and the ~600-byte inclusion witnesses that replace per-transaction receipts entirely (the review's twenty-fourth item, resolved with byte counts); the tokenomics, with the Useful-Work pool right-sized to its advisory role and early contributors raised to 20%; the formal obligations register (theorems A–H) that turns Sections 3–10's informal claims into a specification a prover or auditor can attack; the roadmap and milestones; and the appendices — the full life-of-a-transaction walkthrough, the parameter table, and the governance charter.

Section 11

11. The Witness Model: Receipts Retired, Attestations Amortized

11.1 The Arithmetic That Killed the Per-Transaction Receipt

The review's twenty-fourth item did the multiplication, and the multiplication is fatal: v2.1 issued each user a ~3.5 KB Verifiable Delay Witness. At one million transactions per second that is 3.5 GB of receipt data per second — roughly 302 TB per day — before a single byte of replication. No data-availability layer, no archival market, no engineering heroics make that sustainable.

The first-principles response is to ask what a receipt must actually prove. Three claims, no more: (i) the transaction was valid; (ii) it was included in a finalized block; (iii) its effects — the output notes — exist in the authenticated state. None of these requires a per-transaction cryptographic object produced or stored by the network. Validity is a property the protocol already proves once, in aggregate, as G_τ . Inclusion is a Merkle path. Finality is a committee signature on a root. Effects are the deterministic consequence of an included, valid leg. The receipt reform follows immediately: prove the batch once, attest the batch once, hand the user a path.

11.2 The Three Objects

Object 1 — the recursive batch proof G_τ (Section 5.5). One per interval. A single transparent STARK attesting that every transaction in the registry set T_τ carried a verifying proof. Size 120–180 KB, constant in interval population. Amortized storage at one million TPS: ~0.15 bytes per transaction.

Object 2 — the interval attestation A_τ . One per interval. The beacon committee's ML-DSA quorum certificate over the tuple (interval index τ , G_τ , T_τ root, previous attestation hash, DA roots), where G_τ is the Merkle root over all shard headers of the interval — so one signing event finalizes every shard's blocks, every settled leg, every nullifier insertion, and the registry itself. Each attestation commits its predecessor's hash, forming a hash-chained signature chain: a single trust anchor verifies any suffix. Wire size ≈ 50 KB (15-of-21 ML-DSA-65 signatures; committee keys committed on-chain by hash). Epoch compression: the last attestation of each 24-hour epoch is an epoch attestation over a Merkle root of the epoch's interval attestations, so long-term archival stores one signature set per day and historical verification costs ~50 KB per day of chain — about 18 MB per year — without trusting any checkpoint.

Object 3 — the inclusion witness. Roughly 600 bytes, client-held, never stored by the network, regenerable from archival data at any time. Contents for a typical single-shard transaction:

Element | Size

txid and leg index | 33 B

Merkle path into the block's leg tree (depth ≤ 14 at the 10,000-leg block cap) | ~448 B

Block locator (shard ID, height, interval τ) | ~16 B

Shard header digest | 32 B

Optional: header-to- G path for a verifier tracking no shards (depth 10 at 1,024 shards) | +320 B

Total: ~530 bytes for a wallet that tracks its home shards — as every wallet does anyway, for note detection (§3.2) — and ~850 bytes for a cold verifier that tracks nothing. The “~600-byte witness” quoted throughout this document is the standard case, stated with its cold-verifier variant because a protocol that rounds down is a protocol that will be trusted once.

11.3 What a Witness Proves

W1 — witness soundness (informal): if a witness verifies against a trusted anchor, then the named leg was included in a finalized block of the named shard; that block's header is finalized under the beacon

committee's quorum certificate; and — by state soundness T2 — the block's state update was the exact deterministic application of Update to C_t , so the leg's output commitments are in the note commitment tree and its nullifiers are in the spent set.

Validity is inherited, not re-proved. The leg could settle only under §4.3's rule 1, which required registry membership; the registry's soundness is L1; therefore a verifying witness attests, transitively, the finalized inclusion of a registry-valid leg in a correctly executed block. Six hundred bytes of Merkle path standing on one recursive proof and one attestation per second — that is the entire content of the reform.

Offline verification. A witness verifies offline against any trusted anchor; a fully anchor-free verification walks the epoch-attestation chain to genesis (~18 MB per year of history, one-time, self-contained). This retires v2.1's VDW — whose delay component was, in any event, unspecified — in favor of a finality that is a consensus property rather than a delay property, and whose every byte amortizes.

11.4 Fraud and Light-Client Discipline

A contemporary light-client anchor is one epoch attestation plus the current epoch's interval headers: under 100 KB, verified in ~2 seconds of ML-DSA work (benchmark B6, §13.5). From that anchor, witnesses verify in milliseconds of hashing. A verifier wanting assurance beyond committee signatures samples data availability (§8.7) and spot-checks random shard blocks — which are natively checkable, since every rule of §4.3 is public hashing and set-membership. Full-node re-execution of a shard is cheap by design, and any invalid inclusion is a fraud condition provable from DA data: present the block, present the registry, exhibit the missing membership witness. Fraud proofs slash the producers and the committee signers; the availability guarantee ensures the evidence can always be reconstructed.

11.5 The Amortization Ledger

Item | v2.1 (per transaction) | v3.0 (per transaction, amortized)

Validity proof | inside π (~400 B, Halo2) | ~0.15–0.5 B (bundle tier + G_τ)

Finality receipt | ~3.5 KB VDW | ~0.2 B (A_τ with epoch compression)

Inclusion evidence | inside the VDW | 0 B network-side (client-held ~600 B witness)

Network-side total | ~3.9 KB/tx | ≤ 1 B/tx

At 1M TPS | ~302 TB/day | ~0.1–0.5 TB/day at the protocol tier; witness data exists only where users choose to keep it

The reform removes roughly 99.97% of per-transaction finality-proof bytes from the network. The dominant remaining per-transaction costs are the ones physics insists on — the transaction's own data and its seal — and those are exactly the targets of the compression roadmap in Section 14.

11.6 Portfolio and Batch Witnesses

Witnesses compose. A party proving k legs of the same block shares one locator and header digest, adding ~480 bytes per additional leg: an auditor verifying a 1,000-output settlement in one block holds

~480 KB, not 3.5 MB. A cross-shard payment's user holds one witness per settled leg, bound into one logical receipt by the shared txid (~1.1 KB for the standard two-leg payment). And because witnesses are derivable, loss is a non-event: any archival node can regenerate the witness for any finalized transaction, forever — a property v2.1's bespoke, non-regenerable VDW could not offer.

Section 12

12. Tokenomics: No Pre-Mine, Scheduled Emission, Right-Sized Rewards

12.1 Principles

NERV claims no pre-mine, and this section defines that claim precisely enough to be audited rather than believed:

- Zero spendable supply at genesis. The genesis state commits empty note commitment trees, an empty nullifier set, an empty emission ledger, and nothing else. Not one NERV exists — in any form, in any wallet, in any reserve — at block 0.
- Emission begins at block 1, not before. The first NERV enters existence only through the protocol's deterministic emission schedule, executed by the beacon chain after mainnet launch.
- The schedule is genesis-committed and totals-locked. The full schedule — every bucket's total and pacing — is a genesis parameter. Bucket totals can never be changed; pacing within a bucket's locked total may be adjusted only by constitutional supermajority (Appendix C), and never accelerated in aggregate.
- No discretionary minting, ever. There is no admin key, no treasury mint, no emergency issuance path. The only mints in the protocol's entire state machine are scheduled emission credits and the claim rail of §12.3 — both deterministic functions of finalized public data, both invalid-block-checkable if violated.
- Everything is recomputable. Any full node replaying from genesis can verify that zero NERV existed before block 1, that emitted-to-date equals the schedule exactly, and that circulating supply equals emitted minus burned minus provably abandoned. The emission audit runs in the same conformance CI as Axiom 3's delete test (§13.4).

Fixed total supply: 10,000,000,000 NERV. Token utility is exactly fourfold throughout — fees, validator and market staking, governance weight, and nothing else. This section amends §3.8 accordingly: genesis creates no notes; the entire supply is emitted post-launch (the amendment register at the section's end lists every knock-on edit).

12.2 The Allocation

Category | Share | Prior v3.0 draft | Emission schedule (all post-mainnet) | Rationale

Early contributors | 20% (2.0B) | 20% | 6-month cliff, then linear over 42 months (48-month term) | The v3.0 program — prover market, DKG ceremonies, threshold machinery, formal verification — is multi-year engineering; incentives span it

Founder | 6% (0.6B) | — | 12-month cliff, then linear over 48 months (60-month term); public entitlement ledger; voluntary lockup extensions | Founder continuity across the five-year build program; the longest cliff and longest term in the protocol

Validator & producer subsidy | 30% (3.0B) | 30% | Year-one constant (~545M), then linear decline to zero at month 120 | The security bridge from an empty fee market to a mature one (§12.5)

Community & adoption | 22% (2.2B) | 25% | Sybil-resistant shielded claim windows over 24 months; unclaimed amounts burned at window close | Users first; claims use the custody layer's own machinery (§12.3)

Ecosystem & research | 10% (1.0B) | 12% | Milestone-gated grants over 5 years | Audits (§13.4), the formalization program, leakage analyses (§6.4), cryptanalysis bounties

Foundation operations | 9% (0.9B) | 10% | Quarterly disbursements, declining over 5 years; public reporting | Funds the audit and formalization gates this document promises

Useful-Work pool | 3% (0.3B) | 3% | Era-locked linear over 10 years; payouts only through the out-of-sample gates of §10.4 | The layer is advisory (§10); its pool is sized like one

Total: 100%.

The founder reallocation, from first principles. The 6% founder line is a contributor-class allocation, and contributor-class liabilities are funded from discretionary spending — never from security. It is therefore drawn from the three discretionary buckets — community 25% → 22%, ecosystem 12% → 10%, foundation 10% → 9% — leaving the validator subsidy (30%) and the right-sized Useful-Work pool (3%) untouched. Insider allocations coming at the margin of the airdrop is standard practice; NERV simply says so and keeps the security runway whole. The founder terms are deliberately the strictest in the protocol: the longest cliff (12 months), the longest term (60 months), a fully transparent entitlement ledger, and zero protocol privilege — no admin keys (none exist), no veto, no chamber apart from the ordinary two, governance weight equal to any other note-holder's and no more.

The Useful-Work pool remains 3%, unchanged from the prior draft and for the same reason: the layer is advisory, its failure modes are bounded by design, and a reward pool is a security budget against the failure modes of whatever it pays for. Three percent, era-locked, never a NERV for in-sample loss reduction (P7).

12.3 The Emission Mechanism

Emission is not a policy; it is a deterministic subsystem, built from patterns the protocol already has.

The emission ledger. The beacon chain maintains a BLAKE3-authenticated ledger of emission accounts, committed via an `emission_root` in the beacon state (one field added to the beacon's committed tuple; shard state tuples are unchanged). Each epoch, the beacon deterministically executes the genesis-committed schedule: it credits entitlements to the ledger's accounts and emits the validator subsidy. The ledger has two entry types:

- Signed accounts — for the insider buckets (contributors, founder, foundation, ecosystem grantees): entitlements credit a public ML-DSA identity key, with public amounts and public schedules. Insider transparency is deliberate: the market can price every unlock from public data, forever.

- Commitment-notes — for the community airdrop: eligibility is established pre-launch through sybil-resistant attestation, and the ledger stores a commitment to each eligible claim (a hiding commitment, exactly like a note commitment). Claiming proves knowledge of the opening in zero knowledge and publishes a one-time claim nullifier that permanently marks the entitlement spent. The community claim is, quite literally, a spend of an emission-ledger note — the custody layer's own note/nullifier pattern, lifted one level. Claimant identity and eligibility details remain private; only the aggregate claimed-to-date is public.

The claim rail (notes enter the shielded pool). A recipient converts an entitlement into spendable shielded notes with a claim leg — an ordinary leg settled on the recipient's home shard, whose issue-condition is a beacon-finalized entitlement witness (membership in the emission ledger plus the spent-marker check for commitment-notes, or an ML-DSA signature for signed accounts). Settlement inserts the output note into the NCT and marks the entitlement consumed — double-claim prevention by the nullifier pattern, atomicity by the transit rules (§4.5). Claim legs traverse the mixnet like any other transaction, produce ordinary inclusion witnesses, and are completable by anyone holding the entitlement. This is the same witness-plus-deterministic-mint construction as cross-shard settlement; the protocol adds no new trust to its own money supply.

Validator subsidy. No claim rail is needed: per epoch, the schedule credits each shard's producer payout accounts (the transparent header-chain records of §4.6) directly. Producer compensation at any moment equals fee shares plus the epoch subsidy — both public, both recomputable.

What is public and what is not (P9, stated plainly). Emission-side transparency is the price of verifiable no-pre-mine, and it is paid deliberately: entitlement amounts, schedules, claim events, and claimed-to-date totals are public — that is what makes "no pre-mine" a checkable fact rather than a brochure claim. Everything downstream is ordinary NERV: claimed notes are standard private notes, spendable through the standard machinery, as private as any other. The moment of claiming is public; the money, thereafter, is not.

Emission soundness. One theorem is added to the register of §13: M1 — at every finalized epoch, the emission ledger root equals the exact deterministic replay of the genesis schedule, and every NERV in existence traces to a scheduled entitlement; any over-emission, pre-mine artifact, or discretionary mint yields an invalid block and slashable evidence. M1 extends T3's supply identity to its new form:

$$\text{supply}(t) = \text{emitted}(t) - \text{burned}(t) - \text{provably abandoned}(t)$$

— checkable by anyone, from genesis, forever.

12.4 Vesting Lives at the Emission Layer

A privacy chain cannot enforce transfer restrictions on shielded notes without becoming a surveillance chain, and NERV will not become one. Vesting therefore lives where the protocol can enforce it honestly: at the entitlement layer. A contributor's or founder's tokens do not exist before their cliff — the entitlement

itself does not — so no lock needs policing after the mint. Post-claim, a vested note is an ordinary private note, transferable like any other; the protocol neither can nor should track it.

This is the first-principles answer to insider-lockup mechanics in a shielded ledger: cliffs and linear vesting are enforced by emission timing (protocol-enforceable, zero surveillance), not by transfer locks (which would require the very linkability the custody layer exists to prevent). Where longer insider discipline is wanted, it is obtained by commitment rather than coercion: the founder's entitlement ledger is public, voluntary lockup-extension declarations are published on the same ledger, and note-level time-locks (a spend-after-height field in the note and circuit) are specified as an optional wallet feature on the roadmap (§14) for parties who want self-imposed discipline to be provable.

12.5 The Fee Market and the Subsidy Bridge

The four-way fee split is unchanged: every leg's declared fee divides 40% producer / 30% prover / 20% DA / 10% relay (§12.4 of the prior draft, now §12.5's fee table), funding the same four open markets at the same splits. What changes is the compensation model on top of it: producers now earn fee shares plus the declining epoch subsidy, and the subsidy — not a genesis reserve — is the bridge from an empty chain to a fee-supported one.

The bootstrapping sequence, honestly told. At mainnet, circulation is zero and fees cannot yet fund anything — there is nothing to pay them with. The first NERV in existence are validator subsidies (epoch 1 onward) and the first foundation disbursement; community claim windows open immediately; contributor unlocks begin at month 6; founder unlocks at month 12. Early on, the subsidy dominates producer economics entirely; as circulation grows, the fee share grows and the subsidy declines on its fixed curve, crossing — if adoption arrives — somewhere in the middle years. The B4 proving-economics gate and the B1 throughput gate (§13.5) are benchmarked against the subsidy-era fee floors, so the economics that launch the chain are the economics that were measured.

12.6 The Supply Curve and Burns

Illustrative cumulative emission (percent of total supply, upper bound before burns; the community line assumes full claim rates — realistic claim rates run lower, and every unclaimed NERV is burned at window close):

End of year | Cumulative emitted

1 | ~25%

2 | ~48%

3 | ~62%

4 | ~75%

5 | ~87%

6–10 | tail to 100% (validator subsidy decline + Useful-Work stream)

Three burns, all publicly auditable, all unchanged in mechanism from the prior draft: transparent exits (§3.8), abandoned cross-shard issue legs (§4.5), and unclaimed airdrop entitlements at window close. The per-epoch supply ledger of §12.5 (prior draft) publishes the M1 identity — circulating = emitted — burned — abandoned — with emission now the first term, recomputable by anyone from the emission ledger. The curve is front-loaded enough to bootstrap security and adoption, back-loaded enough (the 10-year validator tail) that the security bridge spans the full fee-market transition, and locked enough that none of it can be renegotiated quietly.

12.7 Security-Budget Honesty (P9)

A no-pre-mine, fixed-supply chain must live on fees eventually; NERV's version of that truth is a ten-year declining subsidy, not a promise that adoption will arrive on schedule. The mitigations are structural and unchanged: validation cost per transaction is unusually low (native hashing, one proof verification per second network-wide), four fee-funded markets share the load, and the emission ledger makes the subsidy's decline a public fact the market can price every epoch. If adoption arrives early, fees replace the subsidy early and the tail is pure margin; if it arrives late, the runway is exactly as long as the schedule says — no shorter, by construction, and no longer, by constitution. The end state, at year ten, is the same end state as every fixed-supply chain: fees or nothing. NERV states it in year zero.

12.8 Governance Chambers and the Bootstrap

The two chambers of the prior draft are unchanged in structure: the note-holder chamber (shielded voting via the note/nullifier machinery — ZK ownership proofs, one-time voting nullifiers, aggregate-only tally reveals) and the validator chamber (transparent stake, ML-DSA votes). Thresholds are unchanged: both-chamber majority for parameters; both chambers plus machine-checked epoch certificates for W-epochs; constitutional supermajority across two consecutive epochs for constitutional amendments — a tier that now explicitly includes any change to emission pacing.

One bootstrapping fact deserves honesty rather than a footnote: at mainnet the note-holder chamber is empty, because no notes exist. Until circulation accumulates — driven by claims and subsidy flow — parameter governance rests with the validator chamber under the constitutional constraints, and every change of consequence (emission pacing, custody semantics, any P-principle) remains constitutional-tier and therefore effectively frozen through the bootstrap window. The founder and contributors hold no special role in this window beyond their eventual, ordinary notes: no chamber of their own, no accelerated weight, no keys. The protocol was designed so that its earliest days require the least trust in the fewest people — and its emission ledger is the proof.

Consistency Amendments (edits required elsewhere in the document)

- §3.8 — Genesis and the Shielded Supply: replace "the entire supply is created at genesis as an initial distribution of private notes" with the no-pre-mine construction: genesis commits empty trees and an empty emission ledger; the entire supply is emitted post-launch per §12; the governance-gated transparent→shielded mint window is superseded by the standing claim rail of §12.3; shielded→transparent burns unchanged.

- §4.2 — the beacon's committed tuple gains emission_root (shard state tuples unchanged; claim legs reference the beacon-finalized ledger by witness, per the transit pattern).

- §4.6 — producer payout accounts receive fee shares plus the epoch subsidy of §12.3; both components public and recomputable.
- §13.2 — add M1 (emission soundness) to the theorem register; restate T3's global identity in the emitted-minus-burned-minus-abandoned form.
- §13.4 — add the emission audit (zero supply at block 0; ledger equals schedule replay) to the conformance CI alongside Axiom 3's delete test.
- Appendix B — replace the "Supply" row ("10,000,000,000 NERV, fixed, no emission") with: "10,000,000,000 NERV, fixed; zero at genesis (no pre-mine); deterministic post-launch emission per §12.2, totals genesis-locked"; add emission schedule rows (cliffs, terms, subsidy curve) to the Economy group.
- Appendix C.1 — the constitutional boundary's "minting beyond the genesis allocation" becomes "minting beyond the genesis-committed emission schedule"; add schedule pacing to the constitutional-tier list in C.2.
- §2.3 and Appendix A — no changes required: claim legs are ordinary legs and appear in walkthroughs only if desired.

The token economy now matches the architecture's deepest commitment: nothing — not the money supply, not the state, not the forecasts — depends on trust in the people who built the chain, and everything they earn arrives on the same public schedule, block by block, after the world can watch it start from zero.

Section 13

13. Formal Obligations: The Theorem Register

13.1 The State Model, Formalized (Review Item 28A)

Per shard s , define: L_s , the finalized block sequence; $S_t = (\text{nct_root}, \text{nullifier_root}, \text{transit_root}, \text{params_root}, \text{prev}, \text{height})$, the authoritative state; $C_t = \text{Commit}(S_t)$, the composite commitment — the sole object with authority; B_t , the canonically ordered settled-leg multiset of block t ; Update , the deterministic function of §4.3 with $C_{t+1} = \text{Update}(C_t, B_t)$; Δ_B , the revealed aggregate (the verifiable decryption of the public ciphertext sum); $e_t = e_0 + \sum \Delta_B \pmod{2^{64}}$, the derived embedding; and $D_t = \text{Commit}(e_t \parallel F_{\text{state}})$, the derived-state root.

Four axioms fix the relationships:

Axiom 1 (Authority). The set of valid state transitions is exactly $\{ (C_t, B_t, C_{t+1}) : C_{t+1} = \text{Update}(C_t, B_t) \text{ and } B_t \text{ is settlement-valid under §4.3's rules 1–6} \}$.

Axiom 2 (Derivation). e_t and D_t are pure functions of public history — finalized blocks and revealed aggregates — recomputable by anyone, consulted by no validity rule.

Axiom 3 (Independence). Deleting all derived data from any prefix of the ledger leaves Axiom 1's transition set unchanged. (The delete test of §7.1, stated as an axiom and enforced as a CI job — §13.4.)

Axiom 4 (Boundness). C_t 's binding strength is that of BLAKE3 over unbounded state; no fixed-dimensional object participates in authority (by E1, none could).

13.2 The Register (Review Items 28B–28G)

ID | Theorem (informal) | Assumptions | Status | Where

T1 | Binding: distinct states with equal commitments require a BLAKE3 collision | BLAKE3 CR (classical 2^{128} ; quantum $\geq 2^{85}$ memory-discounted) | By construction; formal commit-function spec is an obligation | §4.8

T2 | State soundness: every finalized C_{t+1} is the exact Update of a settlement-valid block; induction from C_0 | T1, L1, ML-DSA unforgeability, deterministic execution | Sketch below; formalization in the program | §4.8, §13.3

T3 | Conservation: per-transaction balance in-circuit; global supply identity public and checkable | STARK soundness (QROM), T2 | Per-transaction by circuit; global by induction | §4.8, §12.5

T4 | Persistence: finalized headers immutable without forging $\geq \frac{2}{3}$ committee ML-DSA | ML-DSA, BFT bounds | Standard argument; specification obligation | §4.8

T5 | Cross-shard atomicity: no reachable state mints value; completion is all-or-nothing | T2, transit rules | Obligation: TLA+ state-machine model | §4.5

T6 | Deadlock-freedom: dependency graph acyclic; bounded completion after data availability | Bipartite one-way dependencies | By construction; model-checked | §4.5

T7 | Liveness: issue legs completable by any party, forever | DA availability, registry liveness (§5.5's degraded mode) | Obligation: liveness proof under BFT assumptions | §4.5, §5.5

T8 | Censorship and abandonment honesty: censored spends void cleanly; abandoned issues are provable burns | T5 | By construction | §4.5

E1 | Embedding collision: e is not injective on states — 4,096 bits cannot bind an unbounded ledger | None (information-theoretic) | Proven — pigeonhole | §7.3

E2 | Privacy: adversary knowledge bounded per the property table | ML-KEM, LWE (seal tier), STARK zero-knowledge, BLAKE3 | Leakage analysis is a standing deliverable | §6.4, §14

L1 | Registry validity: every finalized T_τ member carried a verifying STARK | FRI soundness, Fiat–Shamir in QROM, Merkle binding | By construction of §5.5; recursion-soundness audit | §5.6

L2 | Aggregate fidelity: every revealed Δ_B is the exact decrypt of the public sum of proven ciphertexts | Seal well-formedness (statements 9–10), verifiable partials, decode arithmetic | By construction; noise-margin proof obligation | §5.6, §6.3

A1 | Anonymity set bound: expected set $\geq (1-f)^2 \cdot (1-g)^5 \cdot N$ under the stated observer model | Mixnet model | Simulation-validated target (B8) | §6.4

W1 | Witness soundness: a verifying witness implies finalized inclusion of a registry-valid leg in a correctly executed block | T2, L1, T4 | By construction of §11 | §11.3

G | PQ audit: every security-critical primitive hash- or lattice-based; QROM named and bounded | §9.2 inventory | Complete at genesis; standing per-epoch reviews | §9

The review's A–H map onto this document as: A → §13.1; B → T2, L1, W1; C → T2; D → T1, E1, and the commitment inventory of §3.5; E → E2; F → A1; G → §9; H → §13.5 and the milestones of Section 14.

13.3 The Induction at the Heart (T2, Sketch)

The theorem the review's twenty-fifth item demanded — a mathematically strong cryptographic ledger in Layer 1 — deserves its proof sketch stated plainly.

Base. C_0 is the genesis commitment: published, attested by the genesis committee, and reproducible by every full node from the published allocation.

Step. Assume C_t is the exact commitment to the deterministic execution of the finalized prefix. A block B_t is finalized only with a valid ML-DSA quorum certificate; forging one requires $\geq \frac{2}{3}$ of a beacon-sampled committee, excluded under the BFT bound (T4). Settlement validity requires, for every leg: registry membership — which reduces to L1 (STARK soundness under FRI and Fiat–Shamir in the QROM, plus the Merkle binding of T_τ) — plus anchor freshness, nullifier freshness, transit freshness, and issue-conditions, each a public deterministic rule over DA-available data (§8.7). Any full node re-executing $\text{Update}(C_t, B_t)$ therefore obtains the same C_{t+1} ; a header whose committed root disagrees is invalid, and its certificate is slashable forgery evidence. The induction closes: every finalized C is the exact commitment to the deterministic execution of the finalized prefix, and every deviation requires exactly one of three breaks — BLAKE3 (T1), ML-DSA (T4), or the proof system (L1). No learned function appears in the list. No fixed-dimensional projection appears in the list. No hardware, no committee honesty, no forecast appears in the list. That exhaustiveness is what "mathematically strong" means in this protocol, and it is why the neural layer can be deleted — axiom 3 — without touching a single step of the induction.

13.4 The Formalization and Audit Program

Funded from the ecosystem and foundation allocations, gated into the roadmap:

- Machine-checked state model: the transit rules, Update, and T2's induction formalized in Lean 4 — target the state-machine core; stretch goal the full register.
- Model-checked concurrency: T5–T7 under adversarial scheduling, in TLA+.
- Circuit audits: two independent firms on the custody module, the seal chip (statements 9–10), and the aggregation recursion, before mainnet.
- Conformance vectors: golden test vectors for the Adam replay (§10.2), DKG transcripts, decode-and-carry resolution, and witness verification, shipped as a public suite. Axiom 3's delete test runs as continuous integration: the reference client must fully validate a chain with all derived data stripped.

- Cryptanalysis bounties: standing, on the Poseidon2 algebraic margin, the NERV-Seal noise budget, and the (W, B_min) leakage question.

13.5 Benchmark Obligations (Review Item 28H, Answered)

Each benchmark is a pass/fail gate with published methodology, run on public testnets, sequencing the milestones of Section 14:

ID | Benchmark | Pass criterion

B1 | End-to-end sustained throughput, 64 shards, 24 hours | $\geq 250,000$ TPS sustained; inclusion-failure rate $\leq 1\%$; no committee liveness fault

B2 | Latency percentiles | Single-shard p95 ≤ 3 s to finality; cross-shard p95 ≤ 6 s to full completion of all legs

B3 | Wallet proving, device matrix | Mid-range 2025 phone: full bundle p95 ≤ 12 s; laptop p95 ≤ 2 s

B4 | Aggregation economics | ≤ 0.5 GPU-hour per 100,000 transactions, end to end (the §5.5 envelope)

B5 | Decryption ceremony | 512-leg chunk: partials, proofs, and public verification complete ≤ 1 s

B6 | Light-client sync | Contemporary anchor under 100 KB; verified in under 2 s

B7 | Adversarial data availability | Withholding $\geq 20\%$ of samples detected with probability ≥ 0.999 within 30 s

B8 | Anonymity simulation | Measured anonymity sets within $2\times$ of A1's model across (f, g) sweeps at $\geq 10^5$ submissions per window

B9 | Nullifier tree at scale | 10^9 entries: proofs generated ≤ 100 ms, sized ≤ 1 KB

B10 | Epoch rotation ceremony | DKG, reshare, and forced padding of pending batches ≤ 5 minutes; no channel downtime beyond it

Every throughput number in this document remains a design target until B1 says otherwise — and B1 is scheduled before mainnet, not after. A protocol that benchmarks first and launches second is the last discipline this revision owes its reviewers.

The final section delivers the roadmap and milestones that sequence all of it — the audit gates, the benchmark calendar, the compression and folding research tracks that are explicitly not genesis dependencies — followed by the appendices: the complete life-of-a-transaction walkthrough, the consolidated parameter table, and the governance charter.

14. Roadmap, Milestones, and the Honest Frontier

14.1 Sequencing Discipline

The ordering rule governs everything below: audits before benchmarks, benchmarks before launch, launch before scaling. Nothing ships because it worked in a demo; nothing scales because a whitepaper extrapolated it.

Milestone | Window | Contents | Exit gate

M0 — Specification freeze | Q2 2027 | This document; conformance-suite skeleton; the delete-test CI harness (Axiom 3); single-shard reference client | Third-party spec review of §§3–6; CI green on stripped-chain validation

M1 — Cryptographic core | Q3–Q4 2027 | NERV-Seal implementation and DKG; full STARK circuit stack (custody + delta modules); parameter freeze via the lattice estimator; noise-margin obligation of L2 discharged | Two independent circuit audits; estimator report confirming 128-bit classical / 96-bit quantum; golden vectors published

M2 — Testnet zero | Q4 2027 | 8 shards; live mixnet; prover-market bootstrap; epoch-rotation drills | B5 and B10 pass; DKG reshare survives adversarial churn

M3 — Formal program | Q4 2027–Q2 2028 | Lean 4 state model (Update, transit rules, T2 core); TLA+ concurrency model (T5–T7) | Machine-checked induction core; model checker exhausts the adversarial schedule space

M4 — Incentivized testnet | Q1–Q2 2028 | 64 shards; full fee markets; challenger market live; anonymity field study begins | B1–B4 and B6–B9 pass on public data with published methodology

M5 — Mainnet genesis | H2 2028 | 64 shards; genesis allocation per §12; all audit reports public; foundation transparency reporting live | Every gate green; contributor first tranche releases on schedule (§12.2)

M6 — Horizontal scaling | 2029+ | 256 shards upon 30-day sustained B1 at $\geq 2\times$ margin; compression-roadmap deliverables deployed | Both-chamber governance vote plus capacity proof

14.2 Research Tracks — Explicitly Not Genesis Dependencies

Each track states its deliverable, payoff, and the fallback the protocol runs on in the meantime. The discipline stated for folding in §5.5 applies across the board: the chain functions at genesis without any of them, and each fallback is deployed, proven technology.

Track | Deliverable | Payoff | Fallback (status quo)

R1 Folding aggregation | Field-agnostic AIR folding (Bova-class) | $\sim 10\times$ reduction in marginal folding cost | Per-proof recursive FRI verification — proven, deployed technology

R2 Seal compression | Coefficient rounding of v (always-on) and u , CRT splitting | 8–10 KB $\rightarrow$ 4–6 KB per sealed delta | Genesis wire format (§6.3.8); DA-layer placement already absorbs the cost

R3 Ciphertext pruning | Producer-side aggregation proof that ct_B commits to the settled legs' ciphertexts | Individual ciphertexts prunable from DA after settlement | Full ciphertext retention in the DA layer; permanent state is 32 B/tx regardless

R4 Nullifier compaction | Succinct accumulator or period-witnessed compaction | Halts the one super-linear state component | Depth-256 sparse tree — B9-passing at 10^9 entries with ≤ 1 KB proofs

R5 Leakage analysis | Formal treatment of $(W, B_{\min})$ aggregate inference | A parameterized privacy theorem for §6.4 | Standing governance review; $B_{\min} = 128$ with committee padding

R6 Anonymity field study | Longitudinal traffic-analysis study on testnet data | A1's model validated beyond simulation | Simulation-validated targets, honestly labeled as such
R7 Shielded pool contracts | Audited note-model extension points | Programmable privacy | Payments-only at genesis — deliberate (§3.9)
R8 QROM analyses | Tighter Fiat–Shamir-with-abort bounds for the seal's Σ -protocols | Quantified margin | Conservative parameters under the audit posture of §9.3

14.3 The Risk Register

P9 applied to the project itself:

Risk | Honest likelihood | Impact | Mitigation

Wallet proving thin on old devices (B3 margin) | Medium | Adoption friction | Aggregator-assisted proving as an optional service (the wallet blinds its role with range-checked dummy legs); M4 device-matrix data decides

Seal size dominates bandwidth at scale | Certain at genesis | Bandwidth cost | R2 and R3; the budget of §8.8 already carries the genesis number honestly

Prover market concentration | Medium | Centralization pressure — not soundness | Permissionless entry, fee split priced near marginal cost, the B4 gate, staked slashing

Lattice parameter surprises | Low | Seal-channel re-parameterization | The advisory-tier kill-switch of §9.4 — custody never at risk

Aggregation recursion subtleties | Low | Invalid inclusions — fraud-provable, slashable | Dual audits (M1), the 30-second challenge window, degraded-mode attestations

Traffic analysis exceeding A1's model | Medium | Network-wall erosion | Data-layer walls unaffected; R6; relay growth; cover-traffic roadmap

Governance capture | Low–medium | Parameter drift within constitutional bounds | Two chambers, constitutional supermajorities, custody outside every chamber's reach (§C.1)

14.4 Falsifiability

A design that cannot fail cannot be trusted. These results would force redesign, and the protocol says so in advance:

- A formal or demonstrated break of aggregate privacy at (W , $B_{\min} = 128$) beyond the stated computational tier — forces embedding redesign or $B_{\min}$ escalation.
- B3 failure across the device matrix even with aggregator assistance — forces a wallet-tier proof-system rethink.
- Folding cost exceeding the B4 envelope by more than $2\times$ at scale — promotes R1 from research to blocking dependency and delays M6.
- Any TLA+-discoverable livelock in the transit rules under adversarial scheduling — blocks M5.
- A QROM refutation of Fiat–Shamir as instantiated — affects the entire transparent-proof industry; NERV's posture is to migrate to whichever construction survives, at a coordinated upgrade.

Appendix A

A. The Life of a Transaction — Complete Walkthrough

The worked example of §3.6, carried end to end with bytes and timings. Alice (addresses homed to shard 7) pays Bob 50 NERV (address homed to shard 40) and Carol 25 NERV (address homed to shard 7), with fee $F = 10^{-3}$ NERV. Inputs: two notes in shard 7 totaling 60 NERV.

A.1 Construction (client, $t = 0.0\text{--}0.2$ s)

1. The wallet derives delivery addresses for Bob and Carol from their published sets, selecting randomly among Bob's published shards (§8.6 – recipient-side diversification with sender-side randomized choice).
2. Note selection: two inputs ($v_1 = 35$, $v_2 = 25$ NERV), both homed to shard 7; anchor: shard 7's NCT root from a recent finalized header (well inside the 64-header window).
3. Outputs: Bob's note (50 NERV, shard 40), Carol's note (25 NERV, shard 7), Alice's change (9.999 NERV, shard 7 – change homes to the sender's shard by wallet policy).
4. Feature vectors, per leg. ΔS_7 : -60 at Alice's slot, $+34.999$ across the shard-7 outputs' slots, volume rail 34.999, fee share $0.6 \cdot F$, count 1, log rails. ΔS_{40} : $+50$ at Bob's slot, volume 50, fee share $0.4 \cdot F$, count 1, log rails. Admissibility: 3 and 1 active slots – within the limit of 6.
5. Deltas: $\delta_7 = W \cdot \Delta S_7$ and $\delta_{40} = W \cdot \Delta S_{40}$ under frozen W (version tag τ); both checked non-zero client-side.
6. Seals: each δ digitized into 448 ten-bit digits; short randomness sampled by rejection to the in-circuit bounds; ct_7 , ct_{40} computed under the shard-epoch keys from `params_root`.
7. Note encryption: one ML-KEM-768 encapsulation per recipient; ChaCha20-Poly1305 sealing.
8. `txid` = BLAKE3(canonical serialization of both legs).
9. One STARK, both legs, statements 1–11 of §5.1: membership under the anchor, ownership, nullifier derivation, conservation $60 = 50 + 25 + 9.999 + F$ across both legs jointly, ranges, both feature vectors, both deltas, both digit encodings, both seals' well-formedness, and the binding challenge $H(nf_1 \parallel nf_2 \parallel txid \parallel \text{shell digest})$. Proof: ~ 150 KB; proving 0.8–2 s on a laptop, 4–12 s on a mid-range phone, started at signature and overlapped with transit.

A.2 Routing (mixnet, $t = 0.0\text{--}1.8$ s, overlapped)

The transaction (~ 165 KB) is padded into a uniform train of 20 KB packets, onion-wrapped through 5 VRF-selected relays with exponential jitter. Median ingress 1.3–1.8 s. No relay learns sender or contents; the final relay submits to three aggregators simultaneously – censorship insurance (§5.5).

A.3 Aggregation and Registry ($t \approx 1.9\text{--}3.5$ s)

Aggregators verify the STARK natively (~ 0.3 ms), fold it with hundreds of others into a bundle, and submit. The registry committee verifies bundles, resolves duplicates by `txid`, and at the next one-second interval the beacon finalizes T_τ with G_τ . From this instant the transaction is valid protocol-wide. Alice's wallet – watching the beacon – shows first confirmation at roughly 3–4 s from signing.

A.4 Settlement – the spend leg (shard 7, $t \approx 3.5\text{--}6$ s)

Shard 7's producer includes leg 7 after rules 1–6 of §4.3 pass: registry witness valid, anchor fresh, nullifiers nf_1 , nf_2 unseen, transit key unused, no issue-conditions (the leg consumes inputs). Nullifiers insert; Carol's and Alice's output commitments append to the NCT; ct_7 joins the block's ciphertext sum. The header commits the recomputed roots, D_t , $H(ct_B)$, the registry reference, fees, and `QC_hash`. The committee signs; the beacon finalizes.

A.5 Forecast, reveal, and scoring (shard 7, same block)

The header committed $H(\Delta B)$ before decryption. The decryption committee ($t = 7$ of 10) opens the chunk sums with partial decryptions and Σ -protocol proofs; public carry resolution yields Δ_B ; every full node checks the decode against the public sum of the block's proven ciphertexts. The residual $r = \Delta_B - \Delta^*_B$ is recorded; the Huber-blamed, clipped-Adam update commits under $D\{t+1\}$; the embedding advances by 64 additions. No rule of §4.3 consulted any of it.

A.6 Settlement — the issue leg (shard 40, $t \approx 5-9$ s)

Leg 40 creates outputs and consumes none, so rule 5 applies: settlement requires the sibling spend legs finalized. The condition is checked deterministically — a ~ 1 KB witness into shard 7's finalized transit log against a beacon-finalized root. Alice's wallet, or a relay earning the relay fee share (§12.4), submits leg 40 with the witness; shard 40 includes it; Bob's output commitment appends to shard 40's NCT; its own forecast–reveal–score cycle runs; committee signs; beacon finalizes. Bob's wallet, scanning shard 40's encrypted-note stream, trial-decrypts his note.

A.7 Completion

Signing to both legs settled: $p_{50} \approx 6$ s, $p_{95} \approx 9$ s including recipient detection latency (B2's 6 s cross-shard gate measures to leg settlement; detection is gated separately in its methodology). Alice holds two inclusion witnesses (~ 1.1 KB, one per leg, bound by txid). Bob holds none — his note's witness is his ability to spend it, regenerable from archival data on demand (§11.6). Network-side permanent footprint: ~ 0.2 KB.

A.8 The Walkthrough's Moral

Every step reduced to public hashing, an already-verified proof, a set-membership check, modular addition, or a committee signature — with the neural machinery running alongside, consuming public data, committing derived state, and touching nothing. That is Sections 1–13 in one transaction's biography.

Appendix B

B. Genesis Parameter Table

Group	Parameter	Value
Protocol	Supply	10,000,000,000 NERV, fixed, no emission
Protocol	Shards at genesis	64 (6-bit IDs, prefix trie)
Timing	Beacon / registry interval	1 s
Timing	Shard block target	1–2 s within the interval
Timing	Epoch length	24 h (rotation, re-key)
Consensus	Shard committee	21 members, tolerance 7, ML-DSA-65 quorum (15 signatures)
Consensus	Beacon committee	31 members, tolerance 10, staggered thirds
Consensus	Registry committee	21 members, tolerance 7
Custody	Value range	$0 < v \leq 2^{60}$ nano-NEUR
Custody	NCT depth / capacity	32 / $\approx 4.3 \times 10^9$ notes per shard
Custody	Nullifier tree	BLAKE3 sparse Merkle, depth 256
Custody	Anchor freshness window	64 finalized headers

Proofs | System | AIR-STARK, 64-bit Goldilocks-class field, FRI, Fiat–Shamir via BLAKE3
 Proofs | Security | 100-bit PQ soundness; QROM stated
 Proofs | Transaction proof | 120–180 KB; bundle tier amortizes to 250–500 B/tx
 Proofs | Bundle size | 512–4,096 inner proofs
 Overlay | W | 64×256 , 16-bit fixed-point (1.15)
 Overlay | Embedding | $(\mathbb{Z}/2^{64})^{64}$ — 512 B per shard
 Overlay | Spark requirement | ≥ 13 of 224 slot columns
 Seal | Ring / rank | $R_q = \mathbb{Z}_q[x]/(x^{256}+1)$, q a 32-bit NTT prime; rank 8
 Seal | Digitization | 10-bit digits, 448 of 512 slots
 Seal | Committee threshold | $t = 7$ of 10 per shard-epoch
 Seal | Chunk / batch bounds | ≥ 128 ($B_{\min}$) and ≤ 512
 Seal | Ciphertext (genesis wire) | ≈ 8 –10 KB
 Seal | Security tier | ≥ 128 -bit classical / ≥ 96 -bit quantum (advisory)
 Mixnet | Relays per path | 5, VRF-selected
 Mixnet | Packet size | 20 KB uniform
 Mixnet | Jitter | Exponential, $\mu = 100$ ms, $\sigma = 200$ ms, cap 500 ms
 Fees | Splits | 40% producer / 30% prover / 20% DA / 10% relay
 Fees | Genesis prover floor | $\sim 10^{-4}$ NERV per transaction
 Economy | Allocation | 20 / 30 / 25 / 12 / 10 / 3 per §12.2
 Economy | Challenger window | 2,016 blocks (~ 2 weeks), $\geq 90\%$ commit coverage

Appendix C

C. The Governance Charter

C.1 Constitutional Boundary

The P-principles (P0–P9) are constitutional. The custody layer’s validity rules and circuit semantics stand outside every chamber’s legislative reach: no vote, at any threshold, can authorize moving money, minting beyond the genesis allocation, or amending conservation, membership, ownership, or nullification semantics. This is not a norm — it is the construction of Section 3. There is no code path by which governance spends.

C.2 Chambers, Thresholds, Emergency Powers

Chambers and thresholds are as specified in §12.7: both-chamber majority for parameters; both chambers plus the epoch certificate’s machine-checked conditions for W-epochs; both-chamber supermajority across two consecutive epochs plus all outstanding audit gates for constitutional amendments. Emergency powers are enumerated, time-boxed, and expire: the lattice kill-switch (§9.4), hash widening (§9.4), and accelerated split under sustained $2\times$ overload (§8.5).

C.3 The Epoch Ceremony Checklist

1. Freshness beacon transcript published and verified.
2. Candidate W' passes machine checks: quantization format, weight norms, spark ≥ 13 , column–rail independence. Failures void the candidate; the checks are not votable.
3. Both chambers vote; adoption sets an activation height at least 7 days out.

4. params_root commits Hash(W' || version); the forecaster re-initializes at the boundary; the embedding carries forward under the Amnesia Rule.
5. Standing reviews discharged and published: Poseidon2 margin, lattice estimator re-run, the (W, B_min) leakage review (§9.4).

C.4 Transparency Commitments

Quarterly foundation reports (funded per §12.2); published audit reports for every gate of Section 14; the supply ledger of §12.5 published per epoch; all governance transcripts, ballot commitments, and tally proofs archived and verifiable. A protocol that asks to be trusted once is asking too much — this one asks to be checked continuously.

Closing

Four attempts, one principle. The first architecture put a model where the money should have been, and learned nothing it could keep. The second made the mathematics honest and the learning meaningless. The third split the difference beautifully — and still let a 512-byte mirror stand too near the throne. This document's answer to its reviewer is architectural, not rhetorical: the commitment is the truth; the proofs bind to it; the network learns in public beside it; and every wall is built from hashes and lattices, because the era that will stress-test this chain has already begun. The embedding is 512 bytes of honest arithmetic about an economy it will never be allowed to govern — and that, finally, is exactly the right amount of intelligence for a ledger.

Appendix D — Corrections

Version: 3.0-corrigenda

Status: Adopted; folds into v3.1

Source: Third-party recommendations R1–R5 received against v3.0

This appendix records the disposition of five third-party recommendations. All five are adopted; none alters the architecture — the four layers, the canonical pipeline, principles P0–P9, the NERV-Seal design, the sharding model, or the witness scheme. Amendment D.3 is the only item that adds protocol surface (one leg field, one executor rule, and two linear/range constraints). D.1, D.2, and D.5 are normative clarifications, budgets, and assurance gates; D.4 routes an existing sanctioned bridge (§1.4, §8.7) to a new first-line consumer. Parameters introduced here are consolidated in D.6.

D.1 — Proactive Secret Sharing at Decryption-Committee Epoch Boundaries

(Adopted; normative clarification of §6.3. Disposition of R1.)

Rationale. v3.0 already rotates the shard threshold key at every committee epoch and specifies verifiable resharing with forced zeroization of superseded shares (§6.3, §6.3.7). The recommendation asks that this rotation be specified as proactive secret sharing, so that forward secrecy against long-term state compromise is an explicit, stated property rather than an implicit consequence.

Amendments.

1. §6.3 (rotation) is amended to require, at every decryption-committee epoch boundary: (a) establishment of the epoch's threshold key by verifiable resharing to fresh, statistically independent randomness — no share material, seed, or noise is reused across epochs; (b) resharing transcripts with Ajtai commitments and Fiat–Shamir consistency proofs, as in the DKG; (c) mandatory erasure of all superseded share material at the rotation boundary; (d) a bounded handoff window in which the outgoing committee opens any still-pending aggregate before erasure — aggregates unopened at the window's end are recorded as missed reveals, and the forecaster update applies the deterministic skip-and-carry rule (§10), with the miss recorded as public derived state.
2. §6.3.7 is amended by adding the forward-secrecy statement: an adversary who accumulates any number of shares below the threshold t within any single epoch, across arbitrarily many epochs, learns nothing about any epoch's decryption key. Cross-epoch share sets cannot be combined, because each epoch's key material is fresh and resharing re-randomizes shares. The residual is unchanged and stated honestly: t or more colluding members within one epoch can open that epoch's individual ciphertexts; PSS bounds this window to a single committee epoch, and the existing bounds (slashing, stake) apply.
3. §2.5 (seal-committee collusion row) is amended to read: "...bounded by slashing, stake, epoch rotation, and forward secrecy under proactive secret sharing: shares from distinct epochs are non-combinable, and archived or stolen superseded shares are useless by construction."

Honest limits. Erasure of superseded shares is an operational requirement verified by audit, not by protocol. The protocol's guarantee is that the theft of old shares is worthless — the strongest available. Custody was never at risk in any case: the seal carries advisory-tier data only.

D.2 — Proof-Size and Circuit-Constraint Budgets

(Adopted as modified; engineering norm, no architectural change. Disposition of R2.)

Rationale. The recommendation's factual premise is accepted: transparent STARK proofs are tens of kilobytes where Halo2-class SNARK proofs are hundreds of bytes to a few kilobytes. The implied alternative is unavailable to NERV: Halo2 and all Plonk-family systems rest on elliptic-curve assumptions that P1 removes from the security-critical stack, and the dependency policy bans them outright. Size is the accepted, stated cost of transparency, post-quantum binding, and trusted-setup-freedom. What is adopted is the prescribed discipline: strict circuit constraint caps, lookup-table optimization, and explicit per-tier wire-size budgets — which simultaneously bound prover time on mobile clients, the tighter of the two constraints.

Amendments.

1. §5 gains a normative budget table, CI-enforced, initial values governance-adjustable, to be validated against M4 benchmark gates:
 - Whole-transaction AIR: $\leq 2^{22}$ constraints hard ceiling; $\leq 2^{21}$ target. Module ceilings are independent maxima; the whole-AIR ceiling binds their sum.
 - Module ceilings: custody statements $\leq 2^{20}$; delta/encoder $\leq 2^{19}$; seal well-formedness chip $\leq 2^{21}$; BLAKE3 chips $\leq 2^{20}$.
 - FRI security floor: 100-bit post-quantum, unchanged (§5.7).
 - Wallet-tier proof size: ≤ 192 KiB hard ceiling; ≤ 128 KiB target.

- Lookup mandates: 16-bit lookup windows for BLAKE3 (as specified); windowed 60-bit range and 10-bit digit decomposition (as specified); the seal chip's emulated modular arithmetic is required to use lookup tables wherever they reduce constraint count.
- 2. §6.2 is amended with a transport rule: a submitted transaction exceeding one mixnet packet (20 KB uniform) is fragmented across at most $K = 16$ packets, padded to fixed size classes $\{1, 2, 4, 8, 16\}$ packets so packet-count profiles remain coarse; cover-traffic shaping accounts for the class distribution.
- 3. §8.8 (byte budget) is amended to carry per-tier wire sizes explicitly: wallet→aggregator (full transaction, individual STARK included, over the mixnet), aggregator→registry (bundle), registry→chain (G_τ and the T_τ root — unchanged: one proof per interval, $\sim 200\text{--}400$ B amortized on-chain per transaction).
- 4. §14 (roadmap) is amended to rank proof compression and seal-ciphertext compression as the two dominant per-transaction wire objects.

Honest limits. On-chain verification cost is unaffected (aggregation already amortizes it); the binding constraints are peer-to-peer bandwidth and mobile prover time. The ~ 600 -byte inclusion witness (§11) is unchanged. On-device proving remains the privacy-preserving path for mobile wallets; witness-only light flows are viewing, not spending.

D.3 — Transit-Entry TTL and Deterministic Reversion

(Adopted; completes the expiry mechanism already present in the leg format. Disposition of R3.)

Rationale. Legs already carry an expiry height (§3.6) and cross-shard completion is already deterministic (§4.5), but v3.0 does not state settlement semantics at expiry: during a long receiving-shard partition, or if a recipient never claims, committed-but-unminted value would remain pending indefinitely. The recommendation's TTL and reversion semantics are the completion the expiry field implies. One clarification of the recommendation's framing: the transit log never escrows value (§3.4) — it records completion conditions; the lockup risk concerns outputs committed but never minted. The amendment closes that window without a coordinator: the issuing shard reverts deterministically from beacon-finalized data.

Amendments.

1. §3.6 (leg format). Expiry becomes normative with bounds: the issuing leg declares expiry $\in [h_{\text{include}} + T_{\text{min}}, h_{\text{include}} + T_{\text{max}}]$ (initial values in D.6). Any transaction with cross-shard output legs carries, in the issuing leg, one pre-committed revert output per conditional cross-shard output leg: an ordinary note commitment and encrypted note homed to the issuing shard, spendable by the sender. Single-shard transactions are unaffected. Honest byte cost: $\approx +1.3$ KB per cross-shard output leg (one additional ML-KEM-768 encapsulation), recorded in §8.8.
2. §5.1 (statements). The whole-transaction STARK gains two relations, well inside the D.2 budget: (a) the revert equation — $\sum \text{revert-output values} = \sum \text{conditional cross-shard output values}$, with per-leg pairing by leg index; (b) an expiry range check against the anchor window.
3. §3.4 / §4.5 (transit semantics). Transit entries acquire states {Pending, Claimed, Reverted} and are consumed exactly once, by either path:
 - Claim (mechanism unchanged, bounded in time): the receiving shard settles the output leg on an issue-condition witness. A leg whose expiry height is below the receiving shard's current height is invalid at settlement (settlement deadline).

- Reversion (new, deterministic): at any block height $\geq \text{expiry} + L_{\text{grace}}$, the issuing shard's producer must revert every Pending entry past expiry for which the beacon's finalized headers of every receiving shard involved, through height expiry, contain no settlement of the corresponding leg — minting the pre-committed revert outputs and marking the entry Reverted. The rule is a pure function of finalized beacon data; validators check it natively; a block omitting a due reversion is invalid. Wrongful reversion (before the grace boundary, absent the no-settlement condition, or twice) is fraud-provable from public hash evidence.
- 4. Executor rules (§4). A seventh rule is added — expiry reversion as above — and the receiving-shard settlement rule gains the deadline check. Cross-shard header tracking by an issuing shard is bounded by its pending transit entries and the TTL, not by global shard count.
- 5. §13 (theorems). The conservation theorem is restated with the pending term: for any finalized prefix, $\Sigma \text{input value consumed} = \Sigma \text{output value minted} + \text{fees} + \text{value pending in unconsumed transit entries}$; and the pending term is TTL-bounded — no value can remain in transit beyond $T_{\text{max}} + L_{\text{grace}}$. Liveness corollary: reversion requires only the issuing shard, the beacon, and data availability to be live — never the receiving shard.

Honest limits. A recipient whose shard is partitioned past the TTL loses the transfer (the note is never minted) and the sender is made whole by reversion. Wallets and relayer markets therefore race the expiry, and T_{min} is set generously above the normal 2–4 s completion path.

D.4 — Anomaly-Responsive, Bounded Dynamic Fee Floors

(Adopted as modified; consistent with the deterministic-threshold bridge of §1.4 and §8.7. Disposition of R4.)

Rationale. The recommendation correctly identifies a manipulation surface: operational parameters informed by aggregate statistics can be distorted by an adversary who transacts deliberately to move those statistics. v3.0's answer to this class is already constitutional — deterministic threshold rules on finalized public data (§1.4, §8.7), robust statistics for learning (P8), hysteresis for topology (§8.5). The recommendation adds the economic damper, and it is adopted: first-line response to level-2 events becomes an automatic, bounded, self-reverting admission-fee floor, so that sustained distortion costs the attacker at the scaled rate. Two modifications protect the constitution. First, the trigger is evaluated on finalized revealed aggregates — authority-native data interpreted through the frozen codec's rail semantics, which sits inside the authority dependency set — not on forecaster residuals; no authority-side code imports the knowledge layer, and the delete test is preserved. Second, forecaster-relative anomaly alarms remain strictly advisory, exactly as §2.6 states. Nothing in this amendment slashes anyone.

Amendments.

1. §12 (fees) gains the dynamic admission floor: per shard and per interval, $\text{floor} = \text{base_floor} \cdot m$, where $m \in [1, M_{\text{max}}]$ is an exact integer fixed-point function of the shard's trailing window of finalized revealed aggregates, thresholded per rail with hysteresis, decaying toward 1 each interval by factor λ unless re-triggered. A block including a leg whose declared fee is below the floor is invalid — a native integer check on finalized public data; no ZK, no tolerance windows. The fee-then-txid inclusion ordering and the 40/30/20/10 split are unchanged.

2. §8.7 is amended: level-1 events remain ops alerts; level-2 events now trigger the automatic floor escalation, replacing first-line governance review; level-2 persistence beyond one governance epoch escalates to the referendum path, which is retained as override, not first responder.
3. §2.6 ("What NERV does not claim") is amended to add: anomaly-responsive fee floors are bounded ($M_{\max}$) and self-reverting; the residual is stated — an attacker can inflate a shard's admission floor to $M_{\max}$ for the decay horizon, at a cost to themselves proportional to the distortion sustained, and can never touch custody validity, inclusion ordering, or any learned parameter.

Honest limits. The floor is a deterministic control law on economic data, not a learned output; the forecaster never enters it. Governance retains emergency parameter adjustment through its normal slow path.

D.5 — Machine-Checked Verification of the Two Most Safety-Critical Circuit Statements

(Adopted as scoped; assurance gate, zero protocol change. Disposition of R5.)

Rationale. Complete formal verification of the STARK stack is a research-scale program and is not adopted as stated. The recommendation itself scopes the requirement to two properties, and those two are adopted verbatim as mainnet blockers: value conservation (custody-critical — a soundness bug is inflation or theft) and feature-vector admissibility (index-integrity-critical — the property every downstream delta statement presupposes). Both are tractable targets for mechanical proof: integer linear arithmetic and a bounded cardinality constraint.

Amendments.

1. §13.4 is amended: prior to the mainnet branch (M5), machine-checked proofs (Lean 4 or Coq) must establish, against the AIR constraint system as specified: (i) soundness of conservation — any trace accepted by the conservation constraints satisfies $\Sigma \text{ inputs} = \Sigma \text{ outputs} + \Sigma \text{ fees}$ over all legs, together with the D.3 revert equation, under the exact integer semantics of §7.2; (ii) soundness and completeness of admissibility — the constraints accept exactly the feature vectors with ≤ 6 active account slots and in-bounds rails.
2. The extraction gap is stated honestly: the proofs bind the specified constraint system; conformance of the deployed circuit to the verified specification is carried by the frozen conformance vectors, the differential suite (in-circuit vs. native twins), and the dual external audits. FRI and the Fiat–Shamir transcript remain audit-covered, not machine-checked.
3. §14 (roadmap) is amended: post-mainnet extension of mechanical verification to the remaining statements — nullifier derivation, Merkle path chips, seal well-formedness — in priority order.
4. CI: the verified statement set is pinned by specification hash; drift between the proof artifacts and the AIR definitions fails the build.

D.6 — Consolidated Parameter Additions (Appendix B / specs/params.toml)

Parameter	Initial value	Introduced by
PSS cadence	decryption-committee epoch (existing); handoff window = 2 intervals	D.1
Whole-AIR constraint ceiling / target	$2^{22} / 2^{21}$, with module ceilings per D.2	D.2
Wallet-tier proof size ceiling / target	192 KiB / 128 KiB	D.2
Mixnet fragmentation	$K \leq 16$ packets; size classes {1, 2, 4, 8, 16}	D.2
T_min / T_max / L_grace	60 s / 24 h / 10 s	D.3

M_max / λ / base_floor	8 / $\frac{1}{2}$ per interval / governance-set minimum admission fee	D.4
--------------------------------	---	-----

FV gate	spec-hash pinning of verified statements; M5 blocker	D.5
---------	--	-----

D.7 – Register Update

Recommendation	Disposition	Amends
R1 – PSS at epoch boundaries	Adopted	§6.3, §6.3.7, §2.5
R2 – STARK size discipline	Adopted as modified (Halo2 alternative declined under P1)	§5, §6.2, §8.8, §14

R3 — Transit TTL and reversion	Adopted	§3.4, §3.6, §4.5, §5.1, §13
-----------------------------------	---------	--------------------------------

R4 — Anomaly-responsive fee scaling	Adopted as modified (residual-based triggers declined; firewall preserved; no slashing)	§8.7, §12, §2.6
--	--	-----------------

R5 — Formal verification	Adopted as scoped (wholesale FV declined)	§13.4, §14
--------------------------	---	------------